

MOSFET models in LTspice

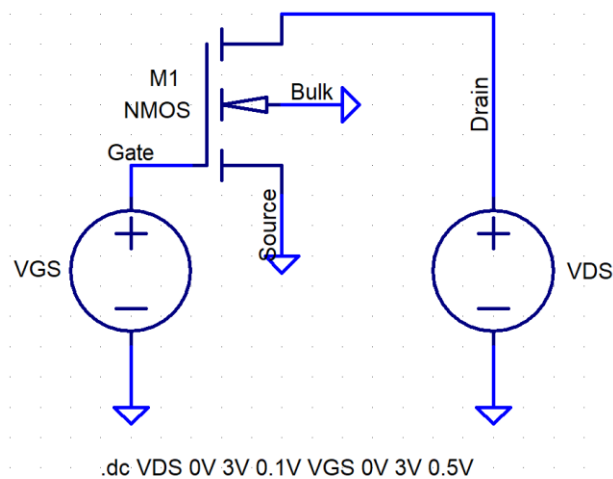
Introduction

Two four-pin MOSFET models are available by default in LTspice. They are stored in the component library under the names *nmos4* and *pmos4*. To the best of our knowledge, these are the only MOSFET models that can be used to simulate CMOS circuits as both models are the only ones in LTspice having separate source and bulk terminals.

Recall that, in CMOS circuitry, the bulk terminal of all N-channel MOSFETs is always connected to ground, whereas that of P-channel MOSFETs is always connected to the supply voltage (V_{DD}). On the other hand, the source terminal of each type of MOSFETs can be connected to either ground or supply voltage depending on the design under consideration. This explains why integrated circuit designers must have the freedom to connect source and bulk terminals to different nodes in their circuit.

Simulations using the *nmos4* model

As an example, you will find below the schematic diagram of a test circuit for the component “*nmos4*”. A DC voltage V_{GS} is applied between gate and source, whereas a DC voltage V_{DS} is applied between drain and source. Both source and bulk terminals are connected to ground. The schematic has been saved in the file named *Test_NMOS.asc*.



The netlist of this circuit is available in the file *Test_NMOS.net* and is as follows:

* C:\Users\Joyce\Documents\LTspice\MOSFETs\Test_NMOS.asc

* Generated by LTspice 26.0.1 for Windows.

VGS Gate 0 *Voltage source, called VGS, placed between the node labelled “Gate” and ground.

VDS Drain 0 *Voltage source, called VDS, placed between the node labelled “Drain” and ground.

M1 Drain Gate 0 0 NMOS L = 500nm W = 1um

*MOSFET 1 with Terminal 1 connected to the node labelled “Drain”, Terminal 2 connected to the node labelled “Gate”, Terminal 3 connected to ground, Terminal 4 connected to ground. Used the Spice model called “NMOS” with parameters $L = 500\text{nm}$ and $W = 1\mu\text{m}$. Recall that L and W are the length and width of our device.

.model NMOS NMOS *Default model for an N-channel MOSFET.

.model PMOS PMOS *Default model for a P-channel MOSFET.

.lib C:\Users\Joyce\AppData\Local\LTspice\lib\cmp\standard.mos *Path to access default model.

.dc VGS 0V 3V 0.1V VDS 0V 3V 0.5V

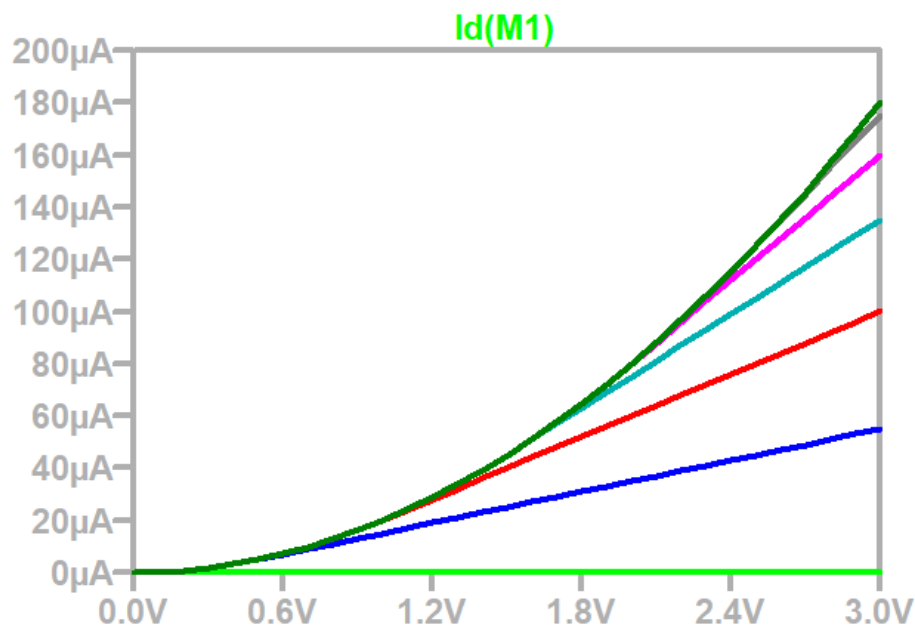
*DC sweep to simulate circuit for $V_{GS} = 0 \rightarrow 3\text{V}$ (step = 0.1V) and $V_{DS} = 0 \rightarrow 3\text{V}$ (step = 0.5V). Here, V_{GS} is the primary sweep and V_{DS} is the secondary sweep. This particular DC sweep provides the transfer characteristic of the MOSFET, i.e., the curves $I_D = f(V_{GS})$ for different values of V_{DS} . Note: This line does not exactly correspond to that shown on the schematic above.

.backanno

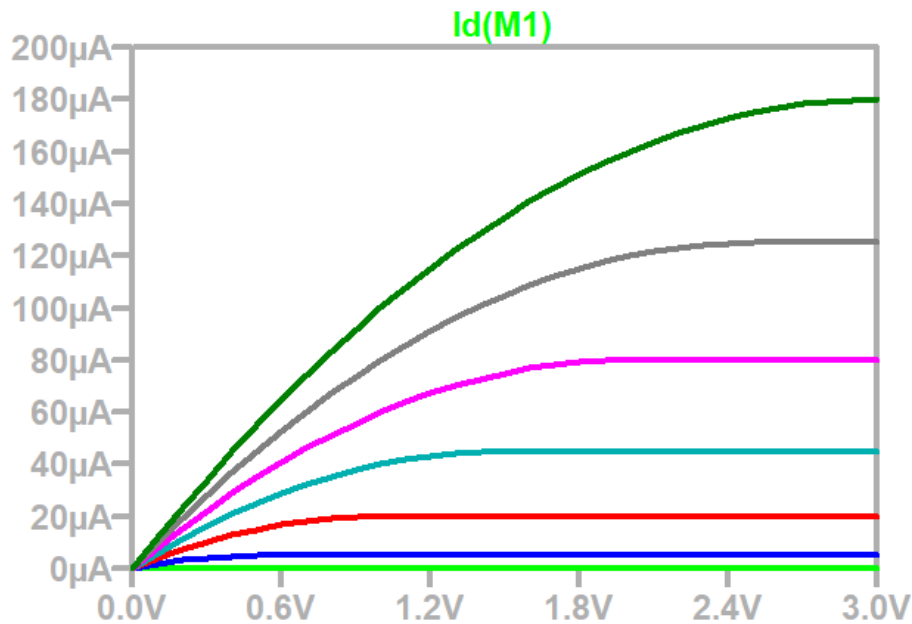
.end

Note: if the simulation does not work, go to Tools → Settings → Netlist Options and make sure that “Default Devices” and “Default Libraries” are ticked. That should solve the issue.

Below, we have plotted the transfer and output characteristics of the *nmos4* component. Recall that the transfer characteristic, shown below, corresponds to the I_D (drain current) versus V_{GS} curve, for different values of V_{DS} .



The output characteristic shown hereafter is the I_D (drain current) versus V_{DS} curve, for different values of V_{GS} .



For those among us with some experience in integrated circuit design, it is perfectly clear that the shapes of those plots indicate that the Spice model used by *nmos4* is quite basic, and thus unrealistic in our era of short-channel devices. In fact, the Spice model used by *nmos4* and *pmos4* for simulations corresponds to the simplest possible set of equations, referred to as the Spice level 1 model.

This is confirmed by two lines (shown below in red) extracted from the log file, called *Test_NMOS.log*, produced by LTspice when simulating the above test circuit with $L = 100\text{nm}$ and $W = 200\text{nm}$:

LTspice 26.0.1 for Windows

Circuit: C:\Users\Joyce\Documents\LTspice\MOSFETs\Test_NMOS.net

Start Time: Fri May 1 21:57:26 2026

solver = Normal

Maximum thread count: 4

tnom = 27

temp = 27

method = trap

Compress1stOrder = true

Compress2ndOrder = true

Instance "M1": Length shorter than recommended for a level 1 MOSFET.

Instance "M1": Width narrower than recommended for a level 1 MOSFET.

.OP point found by inspection.

.step VGS=0

.step VGS=0.5

.step VGS=1

.step VGS=1.5

.step VGS=2

.step VGS=2.5

```
.step VGS=3
Total elapsed time: 0.425 seconds.
Files loaded:
C:\Users\Joyce\Documents\LTspice\MOSFETs\Test_NMOS.net
C:\Users\Joyce\AppData\Local\LTspice\lib\cmp\standard.mos
-----
```

We need a more advanced MOSFET model

It has been known for decades that the Spice level 1 model does not provide a realistic description of any modern MOSFET as found in today's circuits.

To be able to simulate CMOS circuits and obtain realistic results, we need to import new MOSFET models from reliable sources and create two symbols that will be associated with it, one symbol for an N-channel MOSFET and another symbol for its P-channel counterpart. These two symbols will be added to the LTspice library under the *component* category.

This document explains how to do this in a detailed fashion. Here, we are going to focus on the creation of a MOSFET model corresponding to a 45-nm manufacturing process.

Creation of a 45-nm N-channel MOSFET symbol

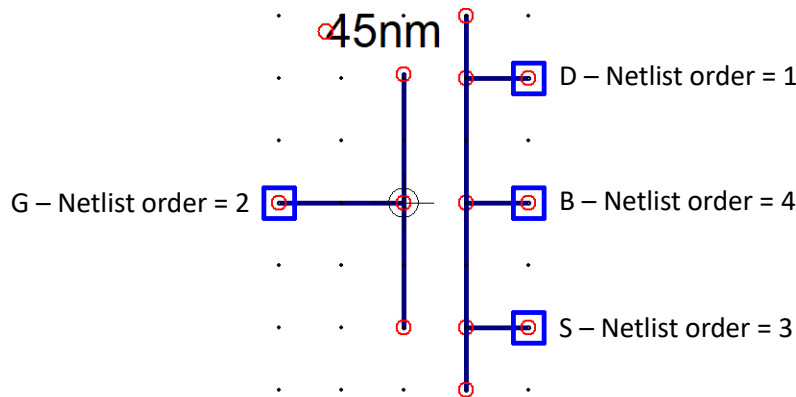
The first thing to do consists of creating a symbol for the N-channel version of this 45-nm MOSFET model.

- **Step 1:** Create a folder for storing all the files needed to define your models.
- **Step 2:** Open LTspice.
- **Step 3:** Go to File → New Symbol, to open the editor in which you are going to draw the symbol for your N-channel MOSFET.
- **Step 4:** Go to Edit → Add Pin/Port, to place the four pins that will connect our MOSFET model to the surrounding circuit.

In MOSFET models, the drain (labelled D for obvious reasons) is in the Netlist order 1, The gate (labelled G) in the Netlist order 2, the source (labelled S) in the Netlist order 3, and the bulk (labelled B) in the Netlist order 4. Note that this is the netlist order, not the label, that will tell LTspice where the drain, gate, source, and bulk are on our symbol.

Once the four pins have been defined this way, connect them in a way that looks like the electrical symbol of an N-channel MOSFET by using the shapes available in Draw.

Your result should look like the one shown below.



Note that this symbol is for now an empty shell as it is not associated with any model yet, i.e. any formal description of its behaviour in a circuit.

Notes from ChatGPT: “In LTspice, the netlist order for a 4-terminal MOSFET is Drain, Gate, Source, Bulk (in that exact sequence) for both N-channel and P-channel devices. LTspice follows the standard SPICE convention for MOSFET terminals. When you create a custom symbol, the pin order must match the order expected by the MOSFET element (M device): Drain (D), Gate (G), Source (S), Bulk/Body/Substrate (B). This corresponds directly to the syntax shown in LTspice documentation: Mxxx Nd Ng Ns Nb <model> ..., where Nd = Drain node, Ng = Gate node, Ns = Source node, Nb = Bulk/body/substrate node.”

• **Step 5:** Go to File → Save As, to save this symbol in the Folder that you created for that purpose. It will be an .asy file by default. You could, for instance, call it *NMOS_45nm.asy*.

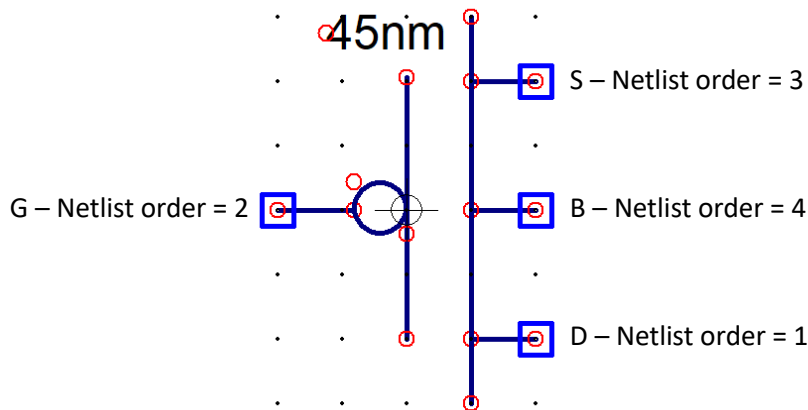
Creation of a 45-nm P-channel MOSFET symbol

• **Step 6:** Go to File → New Symbol, to open the editor in which you are going to draw the symbol for your P-channel MOSFET. Repeat steps 3 to 5 to generate your symbol and save it under the name *PMOS_45nm.asy*.

If you want to save time, you can reuse the *NMOS_45nm.asy* file as a starting point. Save it under *PMOS_45nm.asy* in the same folder and simply add a circle on the gate to create a symbol for your 45-nm P-channel MOSFET.

Your result should look like the one shown below.

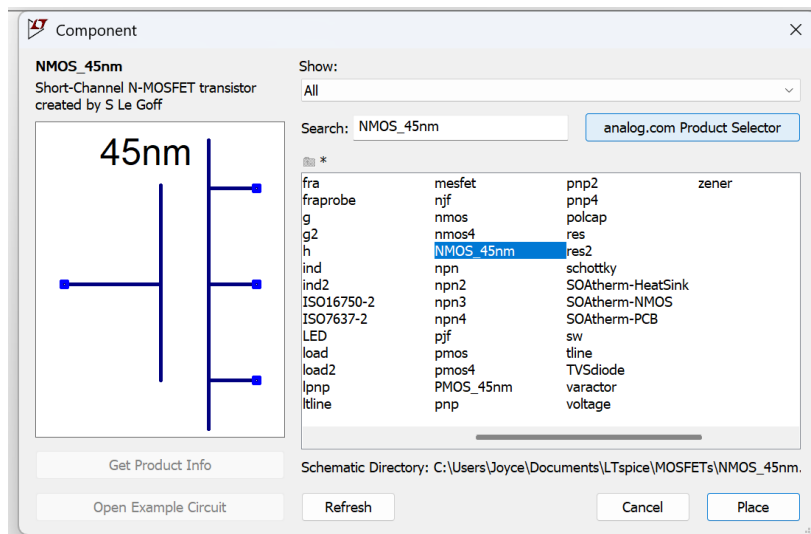
Note that we have now put the source on top and the drain at the bottom. This has been done for reasons of convenience when connecting our components to an external circuit. This point will become obvious later.



At this stage, we have two MOSFET symbols that will act as graphical shells for our models. But one must understand that there is no information embedded into them. So far, LTspice does not even know they are MOSFET symbols.

Assigning Spice models to both 45-nm MOSFET symbols

• **Step 7:** Go to File → New Schematic, to open the LTspice schematic editor. Click on *components*. The two symbols previously created should appear in the selection window. Select *NMOS_45nm* and place it in your schematic editor.



• **Step 8:** Put your mouse on the MOSFET symbol and right click while pressing the ctrl key. The Component Attribute Editor window will then appear.

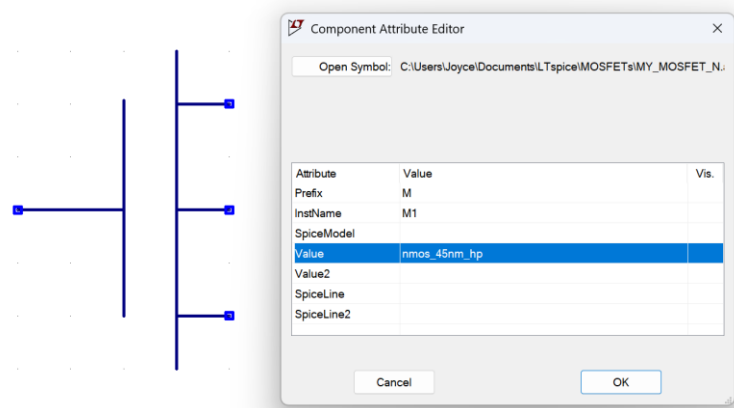
We must fill up some boxes in the Component Attribute Editor as follows:

(1) The symbol's prefix must be set to M. By doing so, you are telling LTspice that *NMOS_45nm* is a MOSFET. Hence, the software now understands that Label 1 of the component must be processed as

the drain, Label 2 as the gate, Label 3 as the source, and Label 4 as the bulk. Things are indeed becoming clearer for LTspice.

(2) The instantiation name (*InstName*) is set to M1 to indicate that this particular component's name is M1, which makes sense as it is the only MOSFET in the circuit. If there is more than one N-channel MOSFET in a circuit, we will have to use M1, M2, M3, etc. to make the distinction between all these instances of the same component.

(3) The symbol's value must match the Spice model's name. We have chosen to call it *nmos_45nm_hp* to avoid any confusion with the preexisting four-pin MOSFET model in LTspice which is called *nmos*. Remember that, so far, we still do not have any file containing this model. We will fix this issue later on.



At this stage, LTspice understands that our four-terminal component is a MOSFET with the name M1 and defined by a Spice model called *nmos_45nm_hp*. However, it still has not been told where to find this model.

Note: These settings are temporary as they have not been saved inside the symbol file itself. In other words, every time a new instance of the *NMOS_45nm* symbol is placed in the schematic editor, the same parameters values (M, M1/2/3/..., *nmos_45nm_hp*) have to be input again in the Component Attribute Editor. This strategy may not always be ideal for circuit designers. To have more permanent settings for our symbol, we will have to rely on the method explained hereafter in Step 10.

• **Step 9:** We can obviously repeat the procedure explained in Steps 7 to 8 to associate a model to the *PMOS_45nm* symbol that we previously created. However, we are hereafter going to show an alternative, and perhaps better, method to do it.

Open the *PMOS_45nm.asy* file in your favourite text editor (e.g., Visual Studio Code or Notepad). Between the CIRCLE and PIN blocks, simply insert the following lines:

SYMATTR Value pmos_45nm_hp

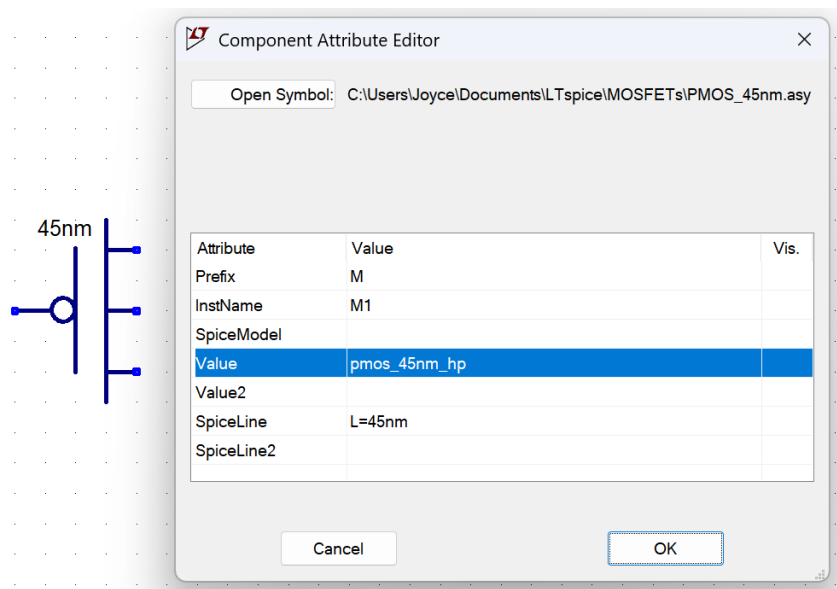
SYMATTR Prefix M

SYMATTR SpiceLine L=45nm

SYMATTR Description 45-nm P-Channel MOSFET created by (your name)

Save the file. Check the result of this action by selecting the *PMOS_45nm* symbol in the component library, placing it in the schematic editor, and ctrl + right clicking on it.

If the window shown below appears, then everything is fine as it means that, after insertion of the above lines in *PMOS_45nm.asy*, LTspice understands that the *PMOS_45nm* component is a MOSFET defined by a Spice model called *pmos_45nm_hp* in which the transistor length has been set to 45nm.



These settings are permanent in the sense that, from now on, every time an instance of the *PMOS_45nm* component is called, the settings will already be defined and entered in the Component Attribute Editor.

In fact, defining permanent settings is precisely what we want to do here. So, before proceeding further, it is important to revisit the *NMOS_45nm* component and insert the following lines in the *NMOS_45nm.asy* text file, between the LINE and PIN blocks:

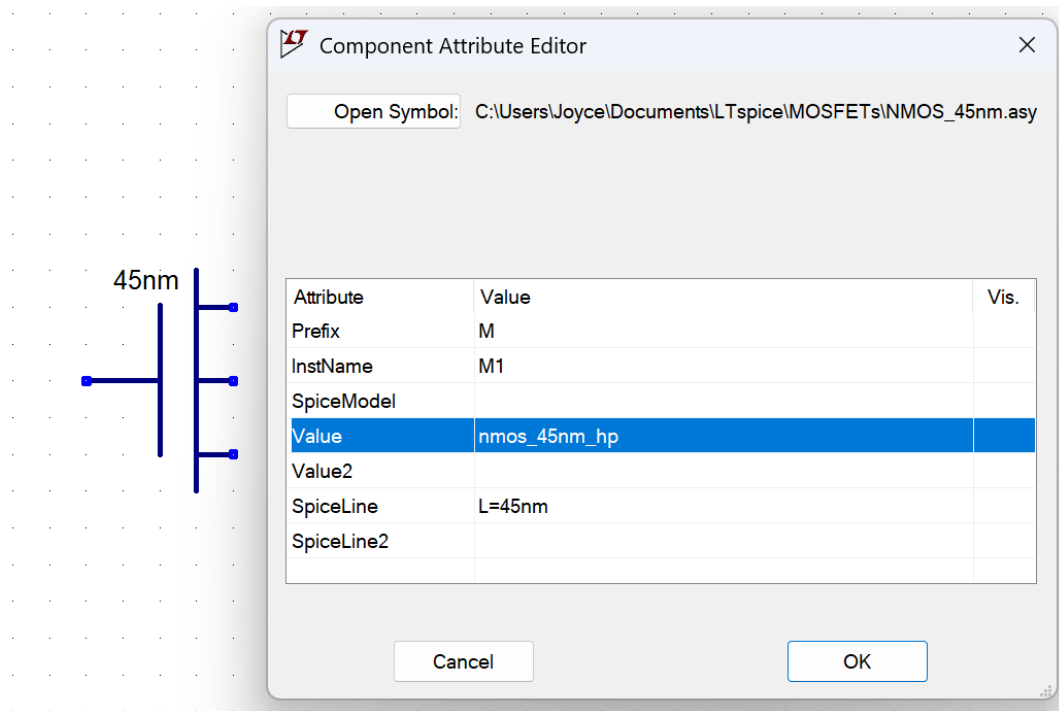
```

SYMATTR Value nmos_45nm_hp
SYMATTR Prefix M
SYMATTR SpiceLine L=45nm
SYMATTR Description 45-nm N-Channel MOSFET created by (your name)

```

Save the file. Do not forget to check the result by selecting the *NMOS_45nm* symbol in the library, placing it in the schematic editor, and ctrl + right clicking on it.

If the window shown below appears, then everything is fine as it means that all *NMOS_45nm* instances will, from now on, be seen by LTspice as MOSFETs defined by a Spice model called *nmos_45nm_hp* in which the transistor length has been set to 45nm.



The final task consists of importing both *nmos_45nm_hp* and *pmos_45nm_hp* Spice models from a reliable source.

Importing a MOSFET model from a reputable source

• **Step 10:** We now need to go online so as to fetch some realistic Spice models for our MOSFETs. For instance, you can find some models at the Microelectronics Co-design Research Group of the University of Minnesota (see <https://mec.umn.edu/ptm> or click on [PTM](#)). You can download any model you wish to adopt in the Bulk CMOS category.

Note that there is no need to download the FinFET models as those will unfortunately not work with LTspice. This is due to the inherent inability of LTspice to handle FinFET models, which is a true pity because these devices are at the core of today's most advanced CMOS circuits. If you wish to perform electric simulations of multi-gate FETs such as FinFETs and GAAFETs, you will have to use a simulator other than LTspice. This task will be left for another day.

You are free to download any model you wish to adopt among the 22nm, 32nm, 45nm, 65nm, 90nm, 130nm, and 180nm options, although we are only going to focus, throughout this document, on the 45-nm manufacturing process. Some of these technology nodes have become obsolete but those with the smallest dimensions are still relevant today.

In any case, these are probably the best options that can be found in the realm of traditional (planar) MOSFETs. In fact, no planar MOSFET technology is viable below the 20nm technology node, which is the very reason why today's most advanced circuits use FinFETs or GAAFETs instead of good old MOSFETs.

Save those model files in the folder where both *NMOS_45nm.asy* and *PMOS_45nm.asy* files were previously saved. Hereafter, we are going to be more particularly interested in the *45nm_HP.pm* file since both *nmos_45nm_hp* and *pmos_45nm_hp* models are embedded in that file.

• **Step 11:** We must make sure that the *45nm_HP.pm* does indeed contain two Spice models called *nmos_45nm_hp* and *pmos_45nm_hp*.

Open the file *45nm_HP.pm* in your favourite text editor. Check the first line after the introductory comments. It should start with “.model”. Whatever appears right after “. model” is the model’s name.

If this model name is already *nmos_45nm_hp*, then everything is fine and you do not need to change anything. If it is something else, then change it to *nmos_45nm_hp*.

```
C: > Users > Joyce > Documents > LTspice > MOSFETs > 45nm_HP.pm
1  * PTM High Performance 45nm Metal Gate / High-K / Strained-Si
2  * nominal Vdd = 1.0V
3
4  .model  nmos_45nm_hp  nmos  level = 54
5
6  +version = 4.0          binunit = 1          paramchk= 1
```

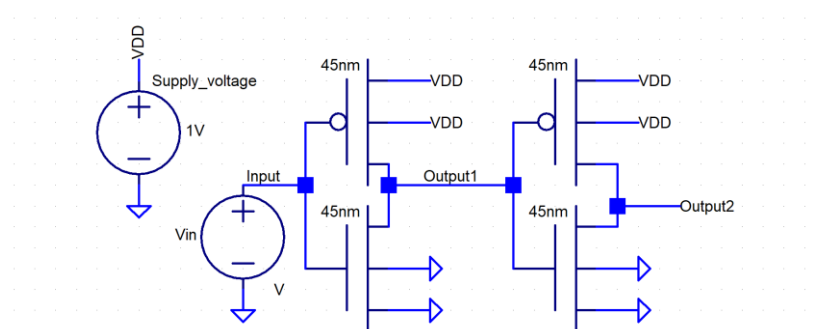
You also need to make sure that the model’s name for the P-channel MOSFET is indeed *pmos_45nm_hp*. You will find the relevant line further down in the file. Make sure that the model’s name is changed to *pmos_45nm_hp* if needed. Do not forget to save the file before closing it.

```
72
73
74  .model  pmos_45nm_hp  pmos  level = 54
75
76  +version = 4.0          binunit = 1
77  learnmod = 2            igmod = 1
```

We now have all the information we need to simulate a CMOS circuit using 45-nm MOSFETs.

Simulating a CMOS inverter using our two 45-nm MOSFET models

• **Step 12:** As an illustration, we can design a CMOS inverter based on the two MOSFET models that we have created. To this end, draw the circuit shown below in the LTspice schematic editor and save it under the name *CMOS_Inverter_45nm*.



Hereafter, we are interested in studying through simulations the behaviour of the first inverter only. To make our simulations meaningful and realistic, this inverter is loaded with another identical inverter. The first inverter is thus said to have a unit fan-out. In practice, the outputs of electronic circuits are indeed connected to other circuits' inputs. Here, the only purpose of the second inverter is to act as a realistic load for the first inverter and thus make our simulation results as meaningful as possible.

Before simulating the circuit, we still need to perform a few more actions.

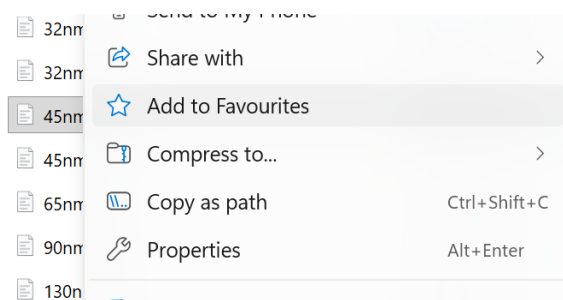
• **Step 13:** We must inform LTspice about the location of the *45nm_HP.pm* file so that it knows where to find the *nmos_45nm_hp* and *pmos_45nm_hp* models.

In the schematic editor, go to Edit → SPICE Directive. A window should then appear. In that window, you can specify the location of the file by typing ".include" followed by the path to *45nm_HP.pm*.

For instance, the author of this document entered the following line:

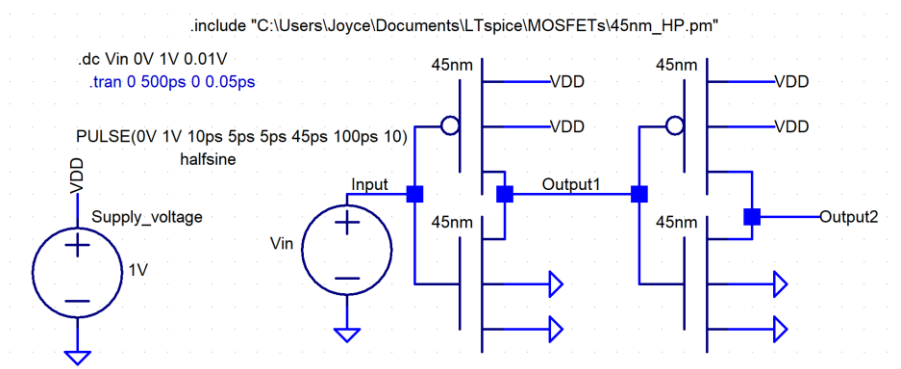
.include "C:\users\Joyce\Documents\LTspice\MOSFETs\45nm_HP.pm"

If you are unsure about the path to your file, just right click on the file and select "Copy as path" as shown below. Then paste the path directly in the SPICE Directive window.



Once it is done, press OK and you should be able to include this SPICE Directive in your schematic.

The circuit should then be similar to the schematic diagram shown below.



• **Step 14:** Finally, to perform a meaningful simulation, you also need to specify, at least, two parameters for each transistor in the circuit. The first of them is the transistor's length *L*. As you probably noticed, the length *L* has previously been embedded in our models so that a designer would

not have to worry about that. For a 45-nm manufacturing process, the length is generally equal to the minimum possible value which is 45 nm. In addition, we must specify the transistor's width W , which is traditionally chosen by the circuit's designer and at least equal to the minimum possible value (45nm).

It is also possible to enter the values of other less important parameters as explained hereafter.

Notes from ChatGPT: "In LTspice, the MOSFET instantiation line (the line that begins with M1, M2 ...) can accept a large set of parameters. Some are required, some are optional, and some depend on the BSIM model you are using (like the PTM 45 nm HP model). Below is the complete, practical list of what you can define.

1. Required parameters - These must appear in the instance line: Drain node, Gate node, Source node, Body (bulk) node, and Model name. Example: M1 D G S B nmos_45nm_hp.
2. Geometry parameters (the ones you care about most) - These are the parameters you typically set for PTM models: L (channel length), W (channel width). AD (drain diffusion area), AS (source diffusion area), PD (drain perimeter), PS (source perimeter), NRD (drain resistance multiplier), NRS (source resistance multiplier). Example: M1 D G S B nmos_45nm_hp L=45n W=1u. If you do not specify AD, AS, PD, PS, LTspice computes default values based on W and L.
3. Multipliers: N (number of parallel devices). Example: M1 D G S B nmos_45nm_hp L=45n W=200n M=4. This behaves like four identical MOSFETs in parallel.
4. Initial conditions – IC=VDS, VGS, VBS (Sets initial voltages for convergence). Example: M1 D G S B nmos_45nm_hp L=45n W=1u IC=0,0,0.
5. Temperature overrides: TEMP (Local device temperature). Example: M1 D G S B nmos_45nm_hp L=45n W=1u TEMP=60

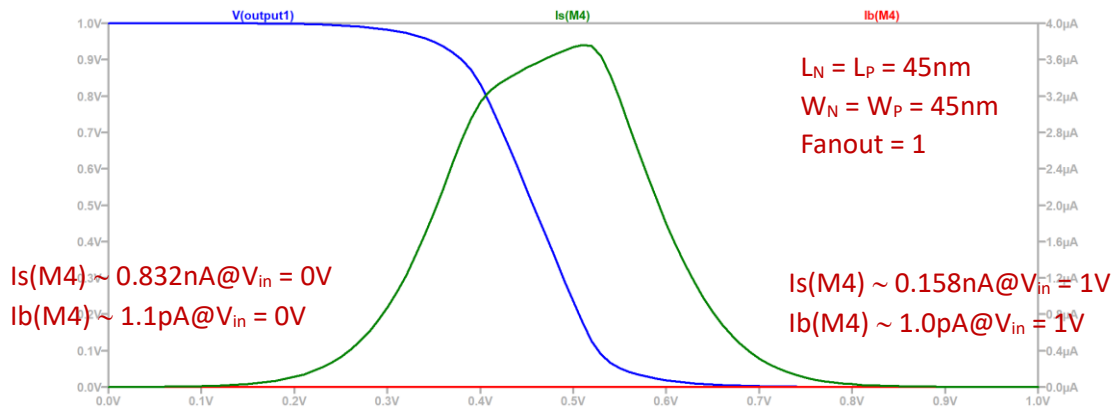
Full example of a detailed MOSFET instance line: M1 D G S B nmos_hp L=45n W=1u AD=0.1u AS=0.1u PD=2u PS=2u M=2 TEMP=27

What you must define for PTM 45-nm models: at minimum, $L = 45\text{nm}$ and $W = ?$ ← you choose this. Everything else is optional unless your circuit requires precision.

You do not need to write the full instance line: M1 D G S B because LTspice automatically generates it for you based on the symbol you placed."

Input the same width $W = 45\text{nm}$ for all four MOSFETs by right clicking on the symbols. Leave the other fields in the window blank.

You can now proceed with the simulation of the inverter which, initially, is going to consist of a DC sweep of the input voltage V_{in} in the range from 0 to 1V with a step of 0.01V. The result, showing the output voltage V_{out} as well as the two DC currents drawn by the first inverter from the power supply, should look like the plot displayed below.



The V_{out} versus V_{in} curve is traditionally referred to as the *DC transfer characteristic*. The shape of this curve is clearly that of a logic inverter.

The currents $I_b(M4)$ and $I_s(M4)$ denote the bulk and source currents of the PMOS transistor of the first transistor, called M4 in the schematic editor. Here, we are only interested in those two currents as their sum corresponds to the total DC current drawn from the power supply V_{DD} by the first inverter.

As can be seen on the above plot, we have measured these DC currents $I_b(M4)$ and $I_s(M4)$ drawn from the power supply for $V_{in} = 0V$ (logic 0) and $V_{in} = 1V$ (logic 1). The measured values allow for the computation of the static power, P_{stat} , consumed by this CMOS inverter using the following expression in which the contribution of the bulk current has been ignored since $I_b(M4) \ll I_s(M4)$:

$$P_{stat} = \frac{V_{DD} \cdot [I_s(V_{in}=0) + I_s(V_{in}=V_{DD})]}{2} \sim \frac{1V \times (0.832nA + 0.158nA)}{2} \sim 0.495nW.$$

The static power consumption of a single CMOS inverter is certainly tiny and may thus seem pretty much insignificant at first glance, but one must remember that modern digital circuits use billions of logic gates on a single silicon chip. A billion times half a nanowatt is, by no means, a negligible power consumption.

The netlist of this inverter circuit is available in the *CMOS_Inverter_45nm.net* file and is as follows:

* C:\Users\Joyce\Documents\LTspice\MOSFETs\CMOS_Inverter_45nm.asc

* Generated by LTspice 26.0.1 for Windows.

Vin Input 0 PULSE(0V 1V 10ps 5ps 5ps 45ps 100ps 10) halfsine

*Voltage source, called *Vin*, placed between the node labelled *Input* and ground. Defined as a PULSE with $V_{initial} = 0V$, $V_{on} = 1V$, $T_{delay} = 10ps$, $T_{rise} = 5ps$, $T_{fall} = 5ps$, $T_{on} = 45ps$, $T_{period} = 100ps$, and $N_{cycles} = 10$, with *Smooth half-sine transitions* ticked. Will be used later on. Redundant for now.

V\$Supply_voltage VDD 0 1V *Voltage source, called *Supply_voltage*, placed between the node labelled *VDD* and ground. Its DC value is 1V.

M3 Output1 Input 0 0 nmos_45nm_hp l=45nm w=45nm

*MOSFET 3 with Terminal 1 (drain) connected to the node labelled *Output1*, Terminal 2 (gate) connected to the node labelled *Input*, Terminal 3 (source) connected to ground, Terminal 4 (bulk) connected to ground. Uses the *nmos_45nm_hp* model with parameters $L = 45nm$ and $W = 45nm$.

M4 Output1 Input VDD VDD pmos_45nm_hp l=45nm w=45nm

*MOSFET 4 with Terminal 1 (drain) connected to the node labelled *Output1*, Terminal 2 (gate) connected to the node labelled *Input*, Terminal 3 (source) connected to the node labelled *VDD*, Terminal 4 (bulk) connected to the node labelled *VDD*. Uses the *pmos_45nm_hp* model with parameters *L = 45nm* and *W = 45nm*.

M1 Output2 Output1 0 0 nmos_45nm_hp l=45nm w=45nm

*MOSFET 1 with Terminal 1 (drain) connected to the node labelled *Output2*, Terminal 2 (gate) connected to the node labelled *Output1*, Terminal 3 (source) connected to ground, Terminal 4 (bulk) connected to ground. Uses the *nmos_45nm_hp* model with parameters *L = 45nm* and *W = 45nm*.

M2 Output2 Output1 VDD VDD pmos_45nm_hp l=45nm w=45nm

*MOSFET 2 with Terminal 1 (drain) connected to the node labelled *Output2*, Terminal 2 (gate) connected to the node labelled *Output1*, Terminal 3 (source) connected to the node labelled *VDD*, Terminal 4 (bulk) connected to the node labelled *VDD*. Uses the *pmos_45nm_hp* model with parameters *L = 45nm* and *W = 45nm*.

.model NMOS NMOS *Redundant statement inserted by LTspice, refers to the default level-1 Spice model for an N-channel MOSFET.

.model PMOS PMOS *Redundant statement inserted by LTspice, refers to the default level-1 Spice model for a P-channel MOSFET.

.lib C:\Users\Joyce\AppData\Local\LTspice\lib\cmp\standard.mos *Redundant statement. Path to access default model.

.dc Vin 0V 1V 0.01V *DC sweep to simulate circuit for *Vin = 0 → 1V* (step = 0.01V).

.include "C:\Users\Joyce\Documents\LTspice\MOSFETs\45nm_HP.pm"

*Path to access *nmos_45nm_hp* and *pmos_45nm_hp* models.

* .tran 0 500ps 0 0.05ps *Transient simulation from 0 to 500ps (step = 0.01V). Will be used later. Redundant for now.

.backanno

.end

To generate a cleaner netlist without the three redundant lines regarding the default level-1 Spice model, just go to Tools → Settings → Netlist Options and untick *Default Devices* and *Default Libraries*. You will then generate the following netlist:

* C:\Users\Joyce\Documents\LTspice\MOSFETs\CMOS_Inverter_45nm.asc

* Generated by LTspice 26.0.1 for Windows.

Vin Input 0 PULSE(0V 1V 10ps 5ps 5ps 45ps 100ps 10) halvesine

V\$Supply_voltage VDD 0 1V

M3 Output1 Input 0 0 nmos_45nm_hp l=45nm w=45nm

M4 Output1 Input VDD VDD pmos_45nm_hp l=45nm w=45nm

M1 Output2 Output1 0 0 nmos_45nm_hp l=45nm w=45nm

M2 Output2 Output1 VDD VDD pmos_45nm_hp l=45nm w=45nm

.dc Vin 0V 1V 0.01V

.include "C:\Users\Joyce\Documents\LTspice\MOSFETs\45nm_HP.pm"

* .tran 0 500ps 0 0.05ps

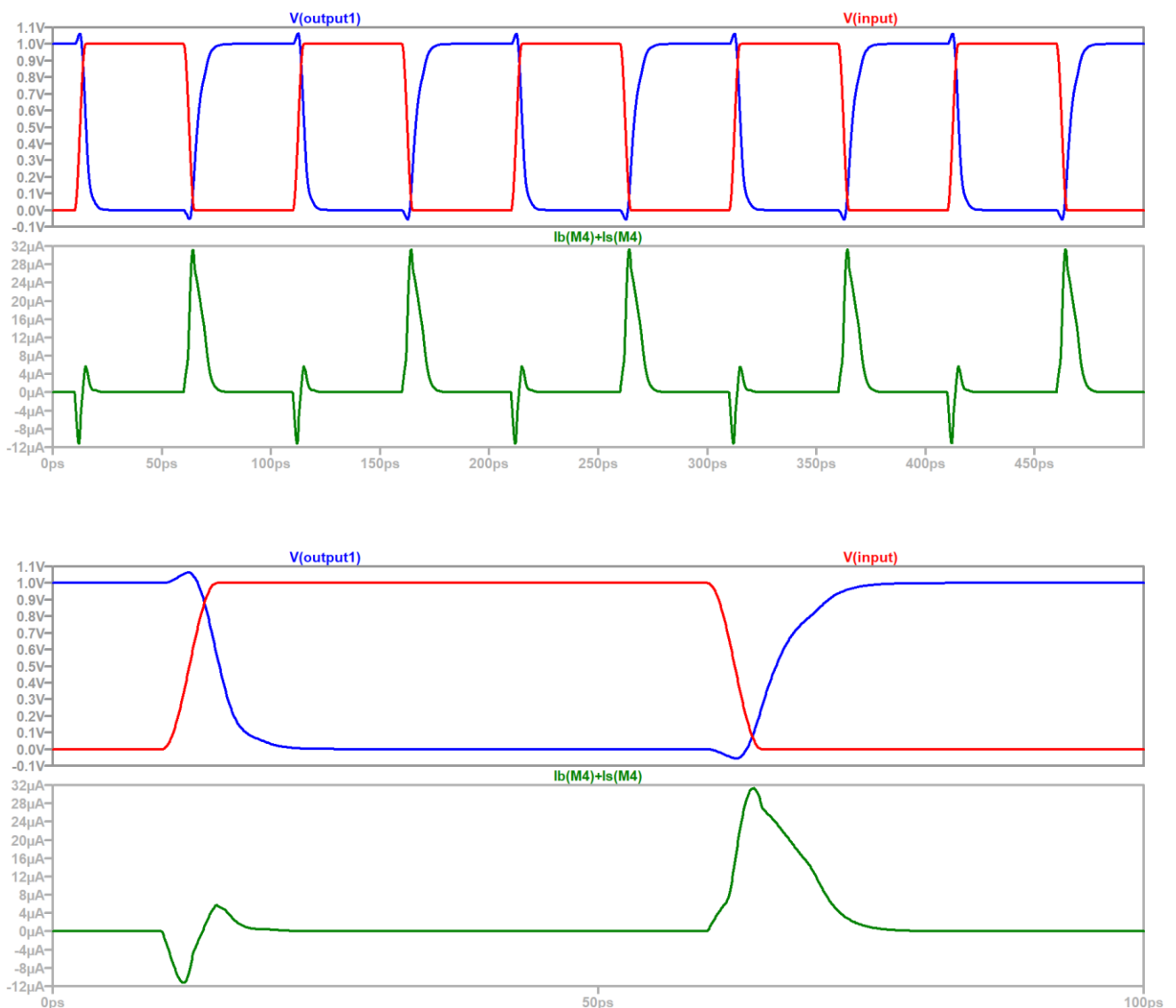
.backanno

```
.end
```

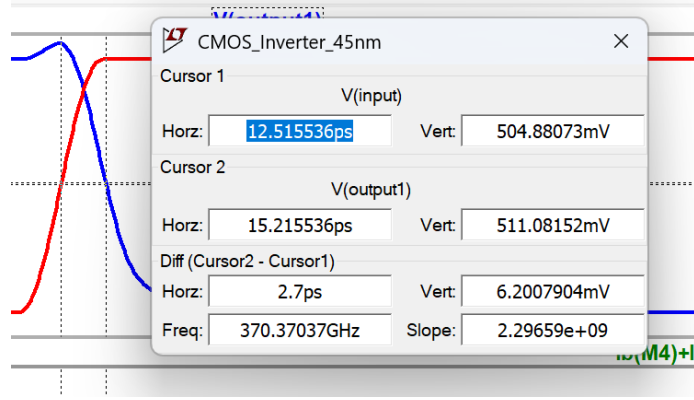
We can also study the inverter circuit in a dynamic setting using a transient simulation instead of a DC sweep.

The time diagram thus obtained is shown below. Note that these simulations results have been generated through the *modified trap* integration method rather than the *trapezoidal* technique as we found that the former does produce slightly more stable results for the currents in this context (but makes no difference as far as the voltages are concerned).

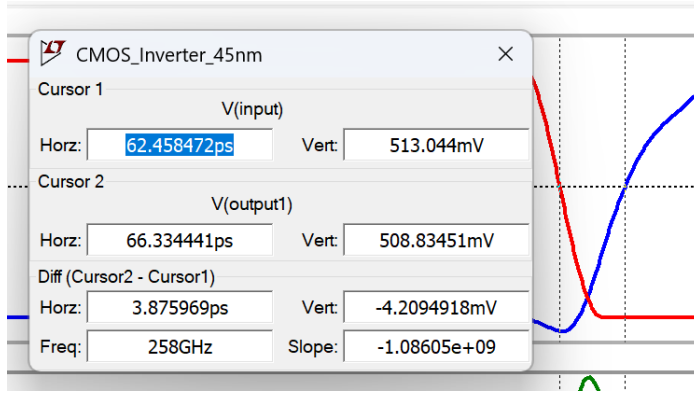
To select the integration method, go to Tools → Settings → SPICE.



Such time diagram is particularly helpful to measure the propagation times from input to output through the inverters. For instance, we can see below that the propagation time from high (logic 1) to low (logic 0) for the first inverter is $t_{phl} \sim 2.7\text{ps}$. Note that such result is valid for a logic inverter with a unit fanout since the first inverter is loaded by an inverter with the same size.



As for the propagation time from low to high for the first inverter with a unit fanout, we measure $t_{plh} \sim 3.9\text{ps}$.



We would obtain higher propagation times by increasing the fanout value (through an increase in the widths of the transistors making up the load inverter and/or by loading the first inverter with two or more same-sized inverters). We could also balance the values of t_{phl} and t_{plh} by increasing the width of the PMOS transistor to compensate for the inherent lower current drive capabilities of the latter compared to its NMOS counterpart.

A time diagram can also be employed to visualise the various currents as well as the instantaneous power consumed by a circuit as a function of time. Here, we are more particularly interested in the evaluation of the total power, $P_{total} = P_{stat} + P_{dyn}$, drawn by a CMOS inverter from the power supply V_{DD} .

We earlier discussed some considerations about the static power, called P_{stat} , and were able to evaluate it using a DC analysis. But, in practice, the crucial parameter is the total power drawn from the power supply which is the sum of the static power, P_{stat} , consumed when the inverter's input and output are constant and equal to one their two possible steady states and the dynamic power, P_{dyn} , consumed when the inverter's input and output are switching.

Recall that the dynamic power consumption is essentially due to the current drawn from the power supply by the PMOS transistor to charge the output parasitic capacitance, C_L , when the inverter's output switches from 0 to 1V. This dynamic power consumption is approximated using the well-known idealized expression

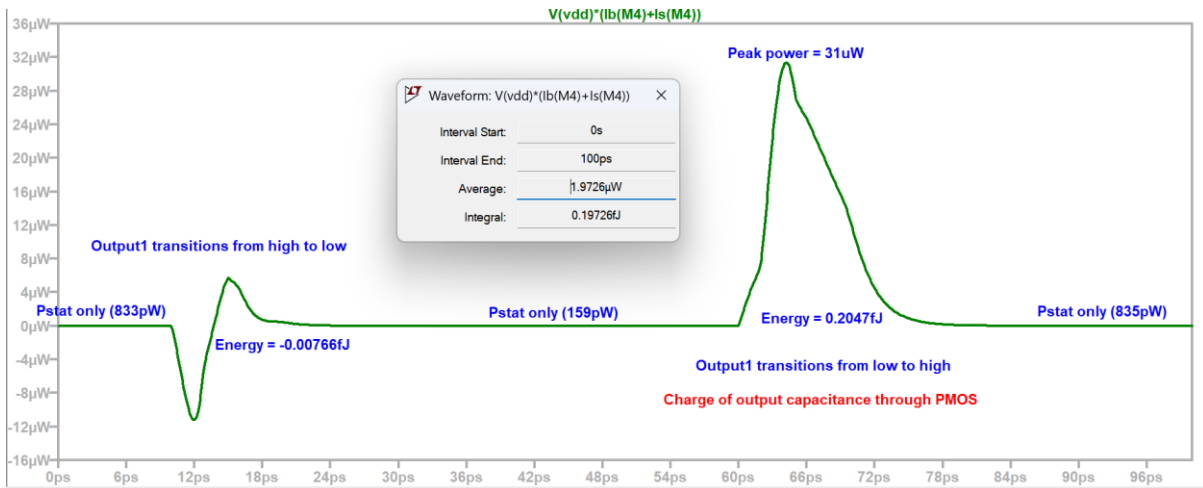
$$P_{dyn} = f C_L V_{DD}^2.$$

The total power drawn from the power supply by the first inverter can be computed in our simulation setting as $P_{total} = V_{DD} \cdot (I_s(M4) + I_b(M4))$, which can, in fact, be approximated as $P_{total} \sim V_{DD} \cdot I_s(M4)$ as $I_s(M4) \gg I_b(M4)$.

With LTspice, it is possible to plot the variation of the sum $I_s(M4) + I_b(M4)$ as a function of time by right clicking on the plot (or, alternatively, going to *Plot Settings*), and select *Add Traces* in the menu that appears. Then, in the *Expression(s) to add* space, type $I_s(M4) + I_b(M4)$.

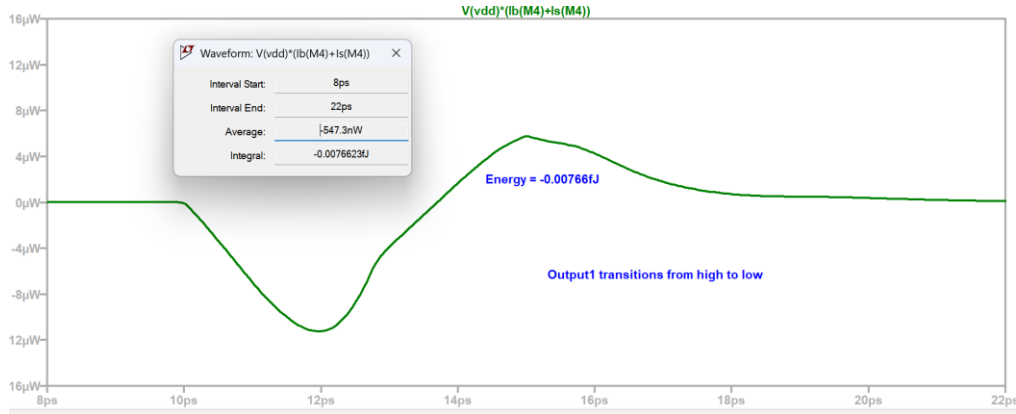
In fact, we can even directly plot power as a function of time, a quantity referred to as *instantaneous power*. Here, we are interested in plotting the instantaneous power drawn from the power supply by the first inverter only. This can be done, by inputting, as explained above, the expression $V(V_{DD}) \cdot (I_s(M4) + I_b(M4))$ in the *Expression(s) to add* space. LTspice understand such expression as being the voltage measured at the node called VDD multiplied by the sum of both currents entering the source and bulk of M4.

You should be able to obtain the plot shown below in which we have added a few explanatory comments.

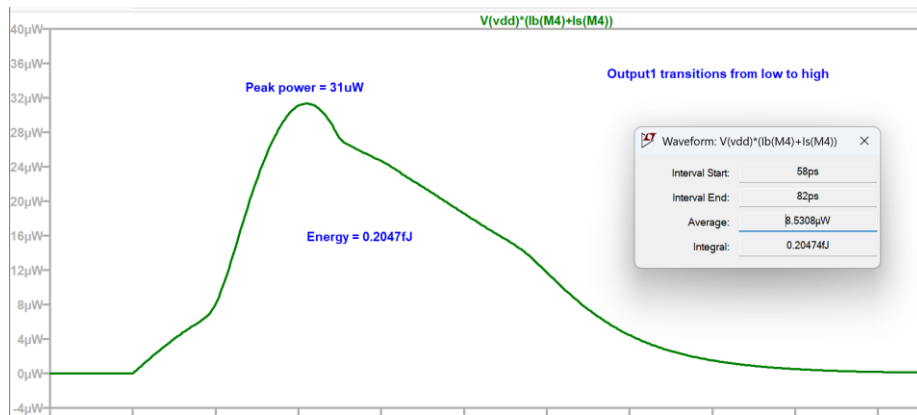


These results indicate that almost all of the power absorbed by a CMOS inverter is due to the charging process of the output parasitic capacitance C_L happening when the output voltage goes from low to high. At the scale of a single inverter, the static power component, P_{stat} , is clearly negligible compared to its dynamic counterpart, P_{dyn} . We can thus write $P_{total} = P_{stat} + P_{dyn} \sim P_{dyn}$.

There is very little power consumed during the time the output voltage goes from high to low. That is because this transition is associated with the discharge of the output capacitance through the NMOS transistor. No current is thus needed from the power supply. In fact, if we think in terms of energy instead of power in order to remove time from our reasoning, the inverter circuit gives back a bit of energy ($\sim 0.00766fJ$) to the power supply voltage when the output voltage goes from high to low.



On the other hand, when the output voltage goes from low to high, significant power is needed to charge the output capacitance. In terms of energy, the circuit takes roughly 0.2047fJ from the power supply to charge the output capacitance C_L . This is when most of the power consumption of our inverter occurs.



Over a cycle including both low-to-high and high-to-low transitions in the input/output signal, the total amount of energy taken from the power supply is given by $\sim -0.00766 + 0.2047 \sim 0.197$ fJ.

As the energy taken from the power supply by our CMOS inverter for completing a full charge-discharge cycle is now known, it is easy to evaluate the total power consumed by this logic inverter at any input/output signal frequency.

In the plots above, the period of the signal is $T = 100$ ps, which corresponds to a frequency $f = \frac{1}{T} = \frac{1}{100 \times 10^{-12}s} = 10$ GHz. At that frequency, the total average power consumption of our CMOS inverter is given by $P_{total} \sim 1.97 \times 10^{-16}J \times 10 \times 10^9Hz = 1.97$ μ W.

At $f = 1$ GHz, we would have $P_{total} \sim 1.97 \times 10^{-16}J \times 10^9Hz = 0.197$ μ W.

In fact, we can see that the total average power is proportional to frequency, which is in line with the idealized equation, $P_{dyn} = f C_L V_{DD}^2$, traditionally used for manual analysis of CMOS inverters.

Creating a CMOS inverter symbol as a subcircuit

We have just shown how to simulate a CMOS inverter. Obviously, from now on, we will also be able to simulate any circuit containing any number of N-channel and P-channel MOSFETs, e.g. amplifier circuits and logic gates. In some more complex designs, the same circuits may have to be reused several times. For instance, we may often have to use a CMOS inverter more than once in a larger design. It thus makes sense to create higher-level symbols that are known as *subcircuits* in LTspice.

We are hereafter going to explore the method to create a CMOS inverter as a subcircuit in LTspice and show how to use it in an example.

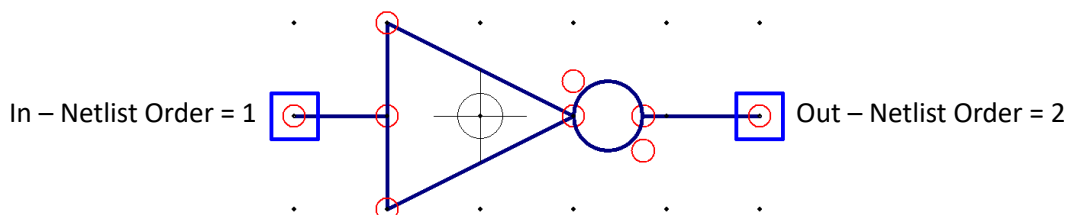
• **Step 15:** Go to File → New Symbol, to open the editor in which you are going to draw the symbol for your CMOS inverter.

Go to Edit → Add Pin/Port, to place the two pins that will connect our inverter symbol to the outside world.

The inverter input is to be labelled *In* and given the Netlist Order 1, whereas the inverter output will be labelled *Out* and given the Netlist Order 2.

Once both pins have been defined, connect them in a way that looks like the logic symbol of an inverter by using the shapes available in Draw.

Your result should look like the one shown below.



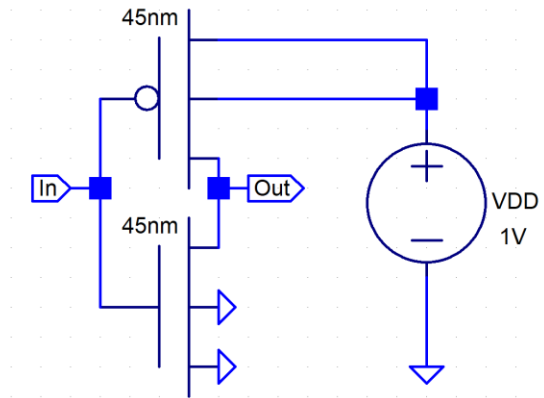
Go to File → Save As, to save this symbol as an .asy file. You could, for instance, call it *Logic_Inverter_45nm.asy*.

• **Step 16:** We now need to draw the schematic that will be associated with the inverter symbol. It needs to have two ports named *In* and *Out* in order to match the notations employed for the *Logic_Inverter_45nm.asy* symbol.

Thus, the *In* port will be defined as an input, whereas the *Out* port will be the circuit output.

You do not have to use a Spice directive for now. The value of $V_{DD} = 1V$ will be defined in this file as our choice is to not have to worry about it later once we work at the logic design level. But, remember that this is an arbitrary choice and it might have been actually wiser to have a second input to our inverter for the power supply V_{DD} as it would have made it possible to evaluate power consumption at a later stage.

Your result should look like the one shown below.

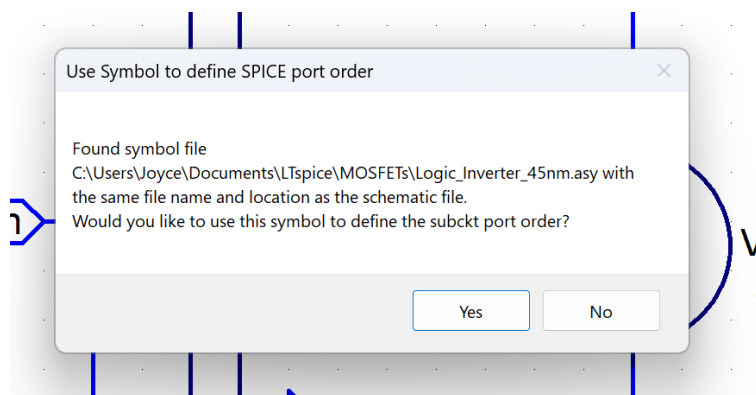


Save this schematic diagram under the same name, *Logic_Inverter_45nm*, as the corresponding symbol file, with an *.asc* extension.

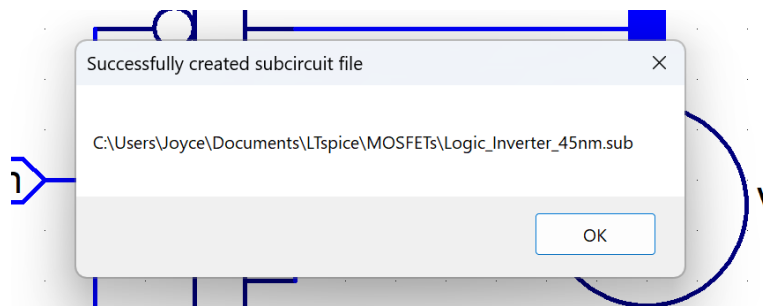
So, at this stage, we have two files that are perfectly compatible since they use the same port numbers and names:

- (1) For the symbol itself: *Logic_Inverter_45nm.asy*;
- (2) For the schematic diagram behind the symbol: *Logic_Inverter_45nm.asc*.

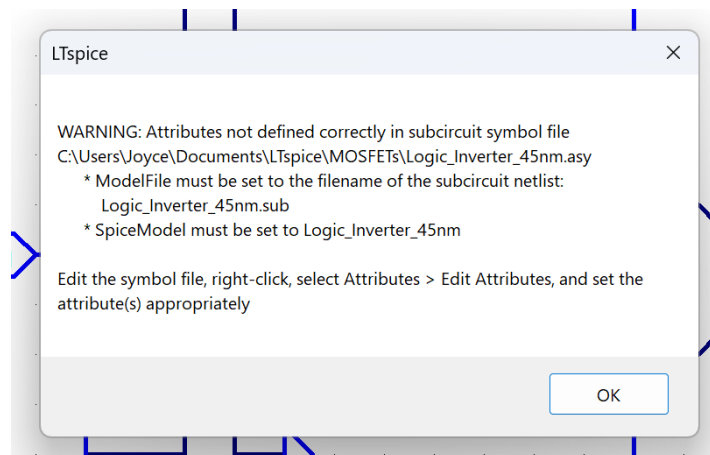
• **Step 17:** We are now going to associate these files. In the schematic editor showing the *Logic_Inverter_45nm* diagram, go to Hierarchy → Generate Subckt Netlist. The window shown below should then pop up.



Click on *Yes*. LTspice should then create a subcircuit (*.sub*) file associated with your symbol (*.asy*) and schematic (*.asc*) files.



Click on *OK*, and a warning message similar to the one displayed below should then appear.



LTspice is telling us the statements that are missing from the symbol file. Thus, make sure that the following lines are present in the *Logic_Inverter_45nm.asy* file, between the LINE/CIRCLE and PIN blocks (although I am unsure whether or not the location of the statements really matters):

SYMATTR Prefix X

SYMATTR ModelFile Logic_Inverter_45nm.sub

SYMATTR SpiceModel Logic_Inverter_45nm

SYMATTR Description Logic inverter in 45-nm CMOS technology with VDD = 1 volt, created by...

The *Logic_Inverter_45nm.sub* file contains the netlist of the *Logic_Inverter_45nm* symbol:

* Copyright (c) 1998-2026 Analog Devices, Inc. All rights reserved.

*

.subckt Logic_Inverter_45nm In Out

M1 Out In 0 0 nmos_45nm_hp l=45nm w=45nm

M2 Out In N001 N001 pmos_45nm_hp l=45nm w=45nm

VDD N001 0 1V

.ends Logic_Inverter_45nm

Improving the CMOS inverter subcircuit

The CMOS inverter that we have just created will work fine. But it has two potential issues, and we are going to fix one of them before proceeding any further.

First, we have not added a second input port to our CMOS inverter to connect the source of the PMOS to V_{DD} . We chose to not do it so as to keep the symbol as *light* as possible. This choice implies that we will not have the possibility to evaluate the power consumption of a circuit simply because there will be no connection through which current can be measured. The whole connection to the power supply will indeed be hidden from the end user.

A bigger issue is that we have fixed in our subcircuit the length L and width W of both MOSFETs. They have been set to $L = W = 45\text{nm}$. For the length, it is perfectly fine to do so because there is no reason to alter its default value in a digital circuit. In contrast, the width is an important parameter that often needs to be adjusted by a designer. What if, in a large and complex circuit, we need to, say, change the widths of one or both transistor(s) inside a particular inverter? With our current CMOS inverter version, this will not be possible. Hence, we clearly need to do something about this problem.

• **Step 18:** Open the *Logic_Inverter_45nm.sub* file with your favourite text editor. Make the following modifications (shown in red) in it and do not forget to save these changes.

```
-----
* Copyright (c) 1998-2026 Analog Devices, Inc. All rights reserved.
*
.subckt Logic_Inverter_45nm In Out Wn=45nm Wp=45nm
* Default values for parameters Wn and Wp.
M1 Out In 0 0 nmos_45nm_hp L=45nm W={Wn}
M2 Out In N001 N001 pmos_45nm_hp L=45nm W={Wp}
VDD N001 0 1V
.ends Logic_Inverter_45nm
-----
```

After doing this, you will be able to specify the width of each transistor individually for every CMOS inverter in your schematic diagram. Those parameter values can be entered in the *SpiceLine* field that appears when right clicking on a particular CMOS inverter symbol in the schematic diagram. In case you decide to leave this field empty, the default values ($W_n=45\text{nm}$ and $W_p=45\text{nm}$) will be used by LTspice.

Notes from ChatGPT: “You don’t have to use the SpiceLine unless you *want* to override the defaults. But you *can* use the SpiceLine if you want each instance to have different W values.

✓ How LTspice handles W parameters in a subcircuit symbol

1. If your .subckt defines defaults

Example: .subckt cmos_inv in out vdd vss Wp=10u Wn=5u. Then, in the schematic, you do NOT need to enter anything in the SpiceLine. LTspice will use the default values ($W_p=10\mu$, $W_n=5\mu$). This is the simplest case.

✓ 2. If you *want* to override W or L for a specific instance

Then you enter the parameters in the SpiceLine of the symbol instance.

Example of SpiceLine: Wp=20u Wn=8u. LTspice merges these with the subckt call: XU1 in out vdd vss cmos_inv Wp=20u Wn=8u. Only the parameters you specify override the defaults. Anything you leave out keeps the default.

✓ 3. If you want the symbol to expose W fields automatically

You can edit the symbol (.asy) and add W_p as a parameter field and W_n as a parameter field. Then the user can type values directly into the symbol properties instead of the SpiceLine. This is optional but very convenient.

✓ Summary

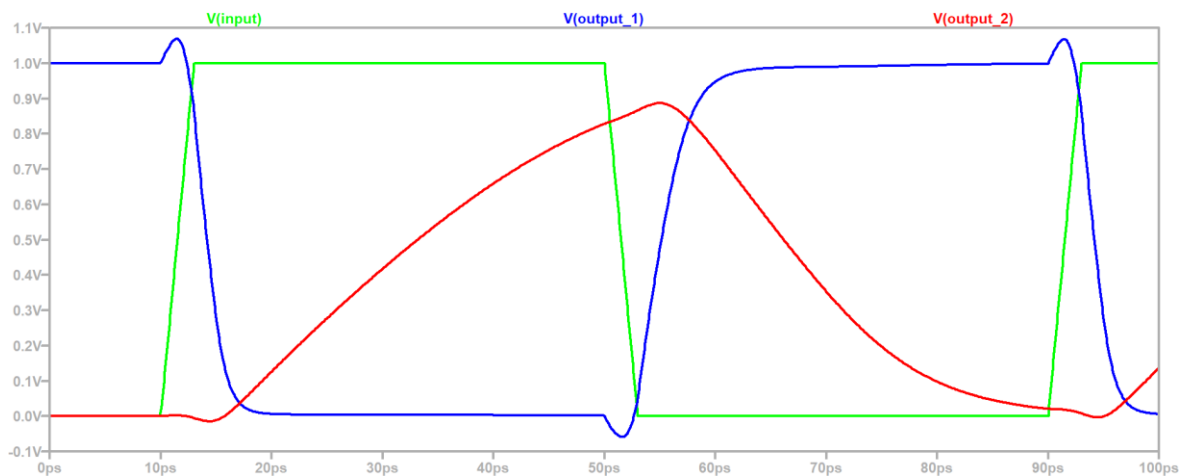
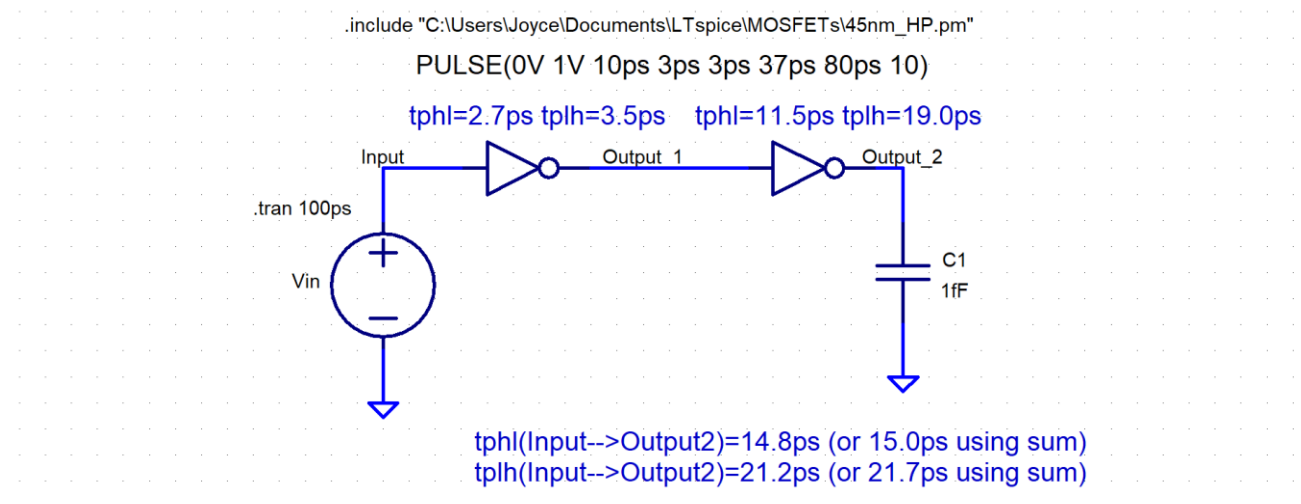
What you want	Do you need to edit SpiceLine?
Use default W/L values	No
Override W/L for one instance	Yes
Make W/L editable in the symbol GUI	No, but recommended to add fields in the .asy file

”

As an illustration, consider the example of the schematic diagram displayed below. The period of the periodic square input voltage is $T = 80\text{ps}$, which corresponds to a signal with a frequency

$$f = \frac{1}{T} = \frac{1}{80 \times 10^{-12}\text{s}} = 12.5\text{GHz}.$$

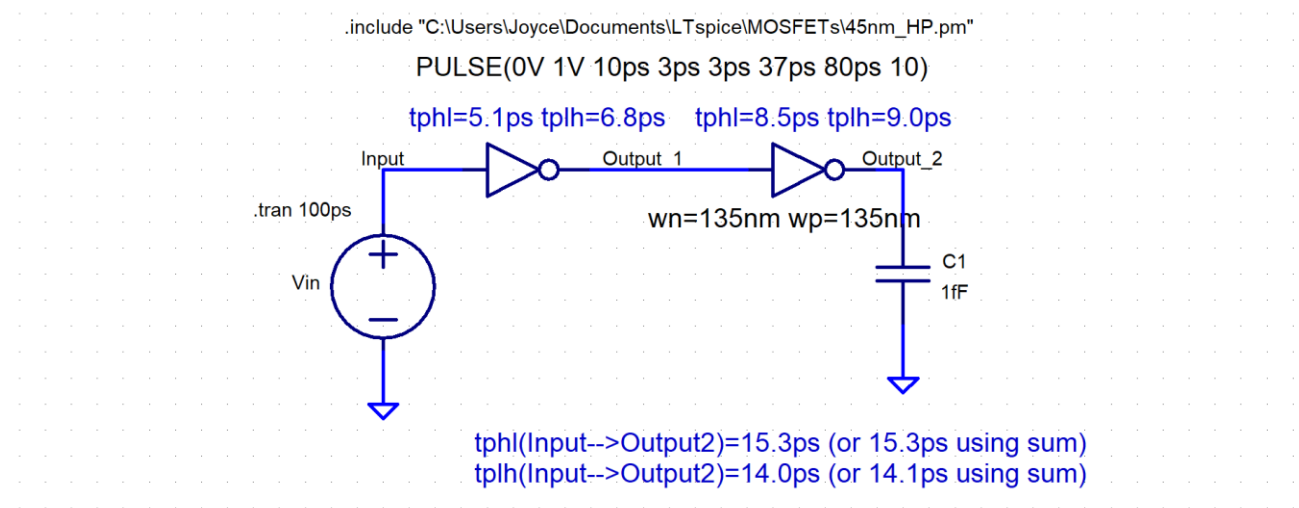
Note that the input voltage is not a perfectly square wave as we have set its rise and fall times to 3ps to make it a more realistic signal. We have measured the propagation times for both inverters; the second one being loaded with a capacitance $C1 = 2\text{fF}$. All transistors have a length $L = 45\text{nm}$ and a width $W = 45\text{nm}$. They are thus minimally-sized.



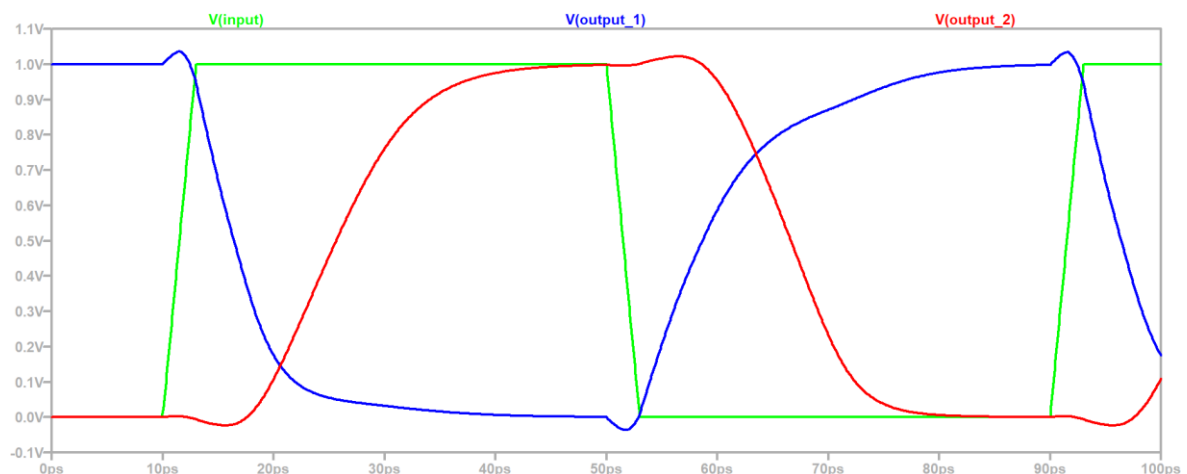
The fan-out of the first inverter is equal to the unit, leading to $t_{phl} \sim 2.7ps$ and $t_{plh} \sim 3.5ps$. The fan-out of the second inverter is unknown but it is certainly rather large as can be seen from its large propagation times $t_{phl} \sim 11.5ns$ and $t_{plh} \sim 19.0ps$. This is not an ideal situation as indicated by the shape of the second inverter's output; this output does not even have sufficient time to fully reach its logic 1 level ($V_{DD} = 1$ volt) before having to start falling down again. Therefore, here, the overall propagation times from input to second output are measured as $t_{phl} \sim 14.8ns$ and $t_{plh} \sim 21.2ps$.

We can improve things by increasing the widths of both MOSFETs in the second inverter, from 45nm to 135nm. This triples the fan-out of the first inverter (as $135nm = 3 \times 45nm$), and thus increases its propagation times from $\sim 2.7ps$ to $\sim 5.1ps$ for t_{phl} and from $\sim 3.5ps$ to $\sim 6.8ps$ for t_{plh} , which is obviously not a step in the right direction.

However, it also offers the advantage of a three-fold reduction in the fan-out of the second inverter, whatever the value of the latter was, significantly decreasing its propagation times from $\sim 11.5ps$ to $\sim 8.5ps$ for t_{phl} and from $\sim 19.0ps$ to $\sim 9.0ps$ for t_{plh} .

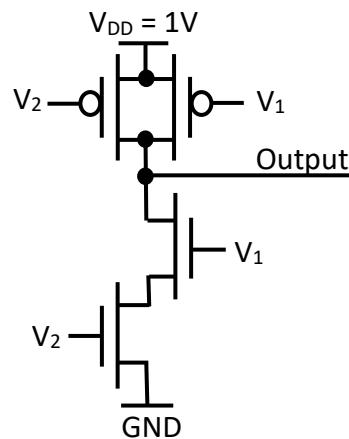


With $W = 135nm$, the overall propagation times from input to second output are measured as $t_{phl} \sim 15.3ns$ (instead of $14.8ns$ for $W = 45nm$) and $t_{plh} \sim 14.0ns$ (instead of $21.2ps$ for $W = 45nm$). The time diagram displayed below clearly shows a significant improvement in the shape of the signal at the second inverter's output compared to the case where all transistors in the circuit had the same width.

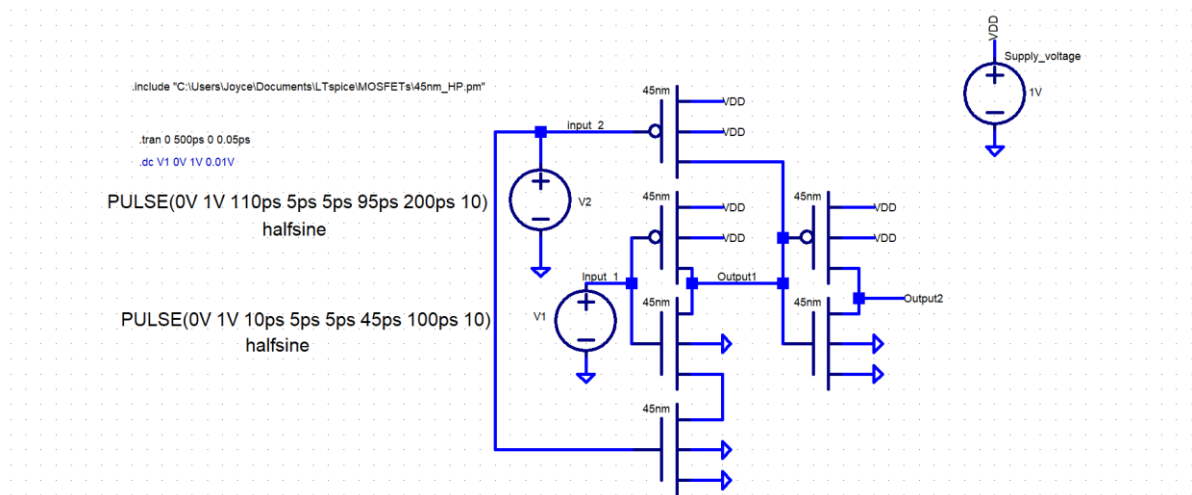


Propagation times of a two-input NAND gate

Let us apply the work we have done to the study the two-input NAND gate in 45-nm CMOS technology. The generic circuit implementing a NAND logic function is shown below. The two NMOS are connected in series between the output node and ground, whereas the two PMOS are connected in parallel between supply voltage and output node.

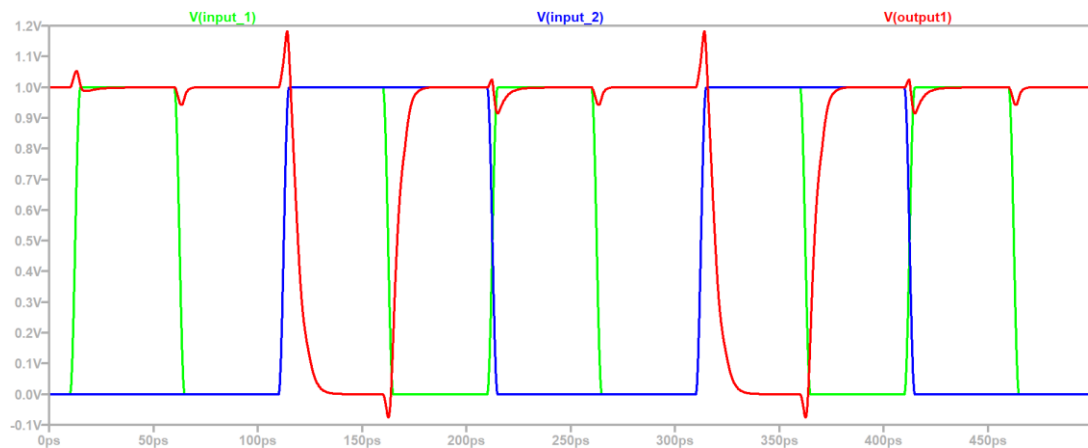


A possible corresponding LTspice schematic is given below, in which the NAND gate's output is connected to a logic inverter's input. The sizes of all transistors in the circuit are minimal, i.e. we have $L = W = 45\text{nm}$ for all transistors. The NAND gate has a unit fan-out as it is loaded with a minimally-sized inverter.



The time diagram below is obtained when performing a transient simulation with input square pulses that have rise and fall times set to 5ps. Such simulation is particularly helpful to study the performance of the circuit in terms of propagation times.

We have thus measured those times by considering all possible configurations leading to a transition at the output node. The results are summarized in the table given below.



Old V1 value	Old V2 value	New V1 value	New V2 value	Transition in NAND output V_{out}	Propagation time
0	0	0	0	-	-
0	0	0	1	-	-
0	0	1	0	-	-
0	0	1	1	$1 \rightarrow 0$	$t_{phl} \sim 6.0ps$
0	1	0	0	-	-
0	1	0	1	-	-
0	1	1	0	-	-
0	1	1	1	$1 \rightarrow 0$	$t_{phl} \sim 5.0ps$
1	0	0	0	-	-
1	0	0	1	-	-
1	0	1	0	-	-
1	0	1	1	$1 \rightarrow 0$	$t_{phl} \sim 6.0ps$
1	1	0	0	$0 \rightarrow 1$	$t_{plh} \sim 2.8ps$
1	1	0	1	$0 \rightarrow 1$	$t_{plh} \sim 4.4ps$
1	1	1	0	$0 \rightarrow 1$	$t_{plh} \sim 5.7ps$
1	1	1	1	-	-

When looking at those results, it is worth making the following comments.

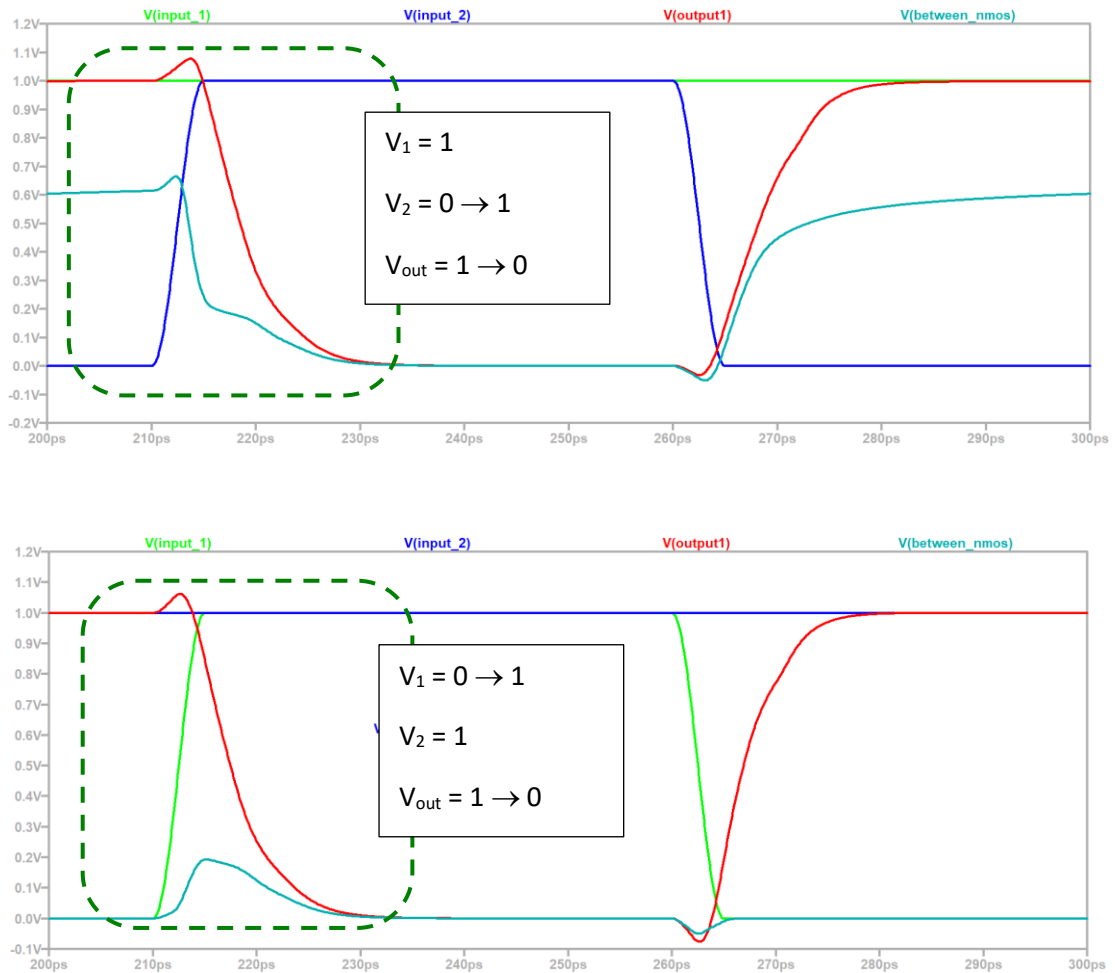
1. The values of t_{phl} are different in the cases $(V_1 = 1, V_2 = 0 \rightarrow 1)$ and $(V_1 = 0 \rightarrow 1, V_2 = 1)$. We indeed have $t_{phl} (1, 0 \rightarrow 1) \sim 6ps$ and $t_{phl} (0 \rightarrow 1, 1) \sim 5ps$.

For simplicity's sake, let us replace, from now on, the notation $(V_1 = x, V_2 = y)$ by (x, y) .

In the case $(1, 0 \rightarrow 1)$, the parasitic capacitance present between both NMOS was connected to the output node, and thus initially charged (not to a full $V_{DD} = 1V$, but to $\sim 0.6V$ only, as an NMOS alone

cannot pass a full V_{DD} value), whereas, in the case $(0 \rightarrow 1, 1)$, this capacitance was connected to ground, and thus already completely discharged before the transition started.

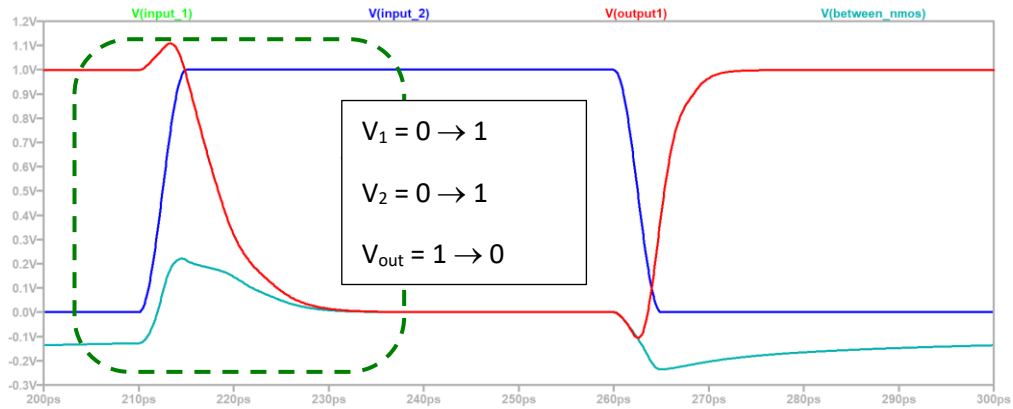
Consequently, this capacitance does not have to be discharged during the transition process in the second case, which shortens the transition's duration compared to the case $(1, 0 \rightarrow 1)$. This is confirmed by the time diagrams shown below.



2. The values of t_{phl} are the same in the cases $(V_1 = 1, V_2 = 0 \rightarrow 1)$ and $(V_1 = 0 \rightarrow 1, V_2 = 0 \rightarrow 1)$. We indeed measured $t_{phl}(1, 0 \rightarrow 1) \sim 6ps$ and $t_{phl}(0 \rightarrow 1, 0 \rightarrow 1) \sim 6ps$.

At first glance, it could indicate that, in the case $(0 \rightarrow 1, 0 \rightarrow 1)$, the parasitic capacitance between both NMOS, which was left floating when both NMOS were off, was initially charged to a value of $\sim 0.6V$, as in the case $(1, 0 \rightarrow 1)$.

In fact, the time diagram below indicates that it is more complicated than that since the floating node is at a voltage $\sim -0.13V$ just before the transition occurs and jumps to a peak value of $\sim 0.22V$ at the start of this transition following the rises in both input voltages. It eventually falls back to zero volt, following the normal discharge of this node, but this behaviour has slowed down the output transition.

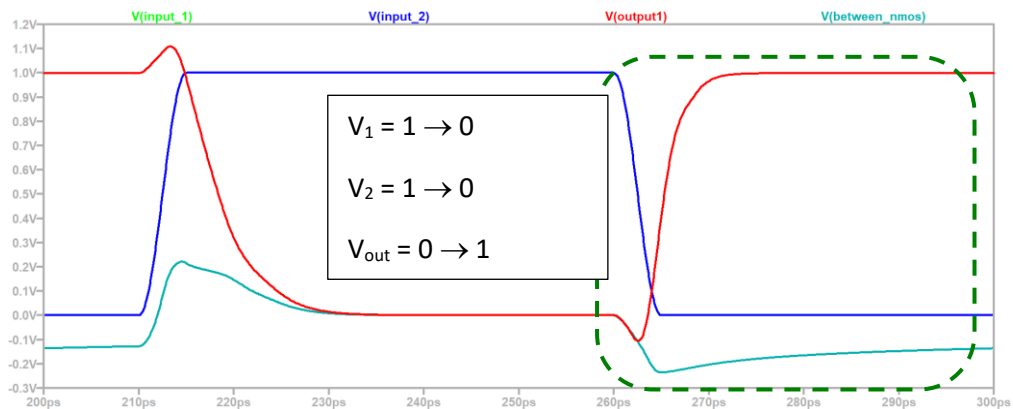


Let us denote as X the node between both NMOS transistors.

The node X is connected to the drain of the bottom NMOS and the source of the top NMOS, whereas the NAND inputs are connected to the gates of both NMOS. So, X is linked to the gate inputs V_1 and V_2 through two parasitic capacitances, namely the capacitance C_{gd} between gate and drain of the bottom NMOS and the capacitance C_{gs} between gate and source of the top NMOS.

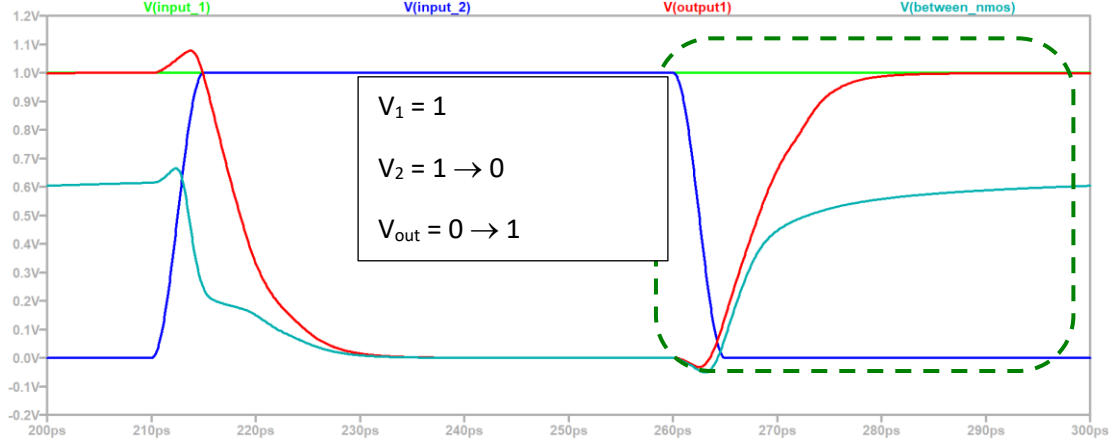
Since the voltage across a capacitance does not change instantaneously, the voltage at node X tends to follow the rapid rises and falls in V_1 and V_2 . Following a $(0 \rightarrow 1, 0 \rightarrow 1)$ transition, the voltage at X goes progressively to zero as this node is then connected to ground through the bottom NMOS that is in the on state.

However, after a $(1 \rightarrow 0, 1 \rightarrow 0)$ transition, the voltage at X that reached $\sim -0.2V$ at the end of this transition, as it followed the gate voltage's rapid fall, remains more or less constant as X is now floating without any significant discharge path.

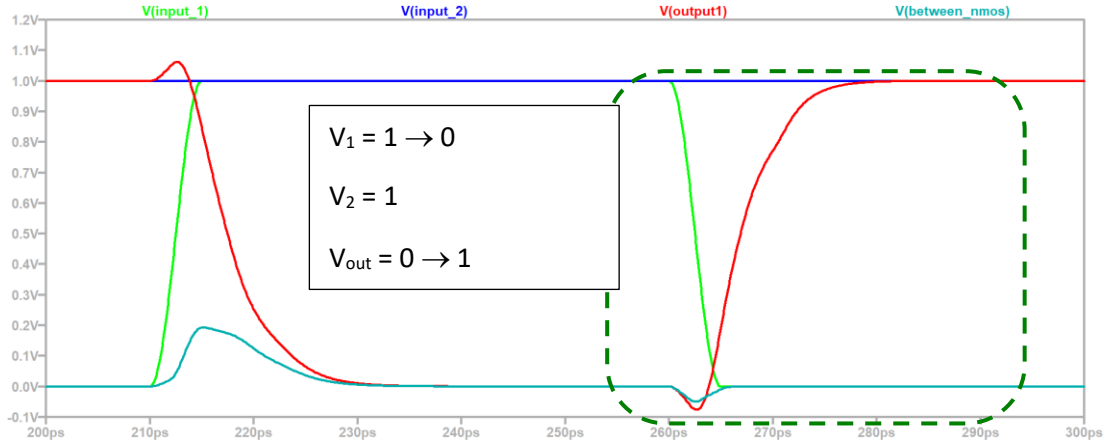


3. The values of t_{plh} are different in the cases $(V_1 = 1, V_2 = 1 \rightarrow 0)$ and $(V_1 = 1 \rightarrow 0, V_2 = 1)$. We have $t_{plh}(1, 1 \rightarrow 0) \sim 5.7ps$ and $t_{plh}(1 \rightarrow 0, 1) \sim 4.4ps$.

In the case $(1, 1 \rightarrow 0)$, the top NMOS transistor is always connected to the output node. Therefore, when this node has to switch from 0 to 1, the parasitic capacitance at node X (between both NMOS) must be charged alongside the output capacitance.

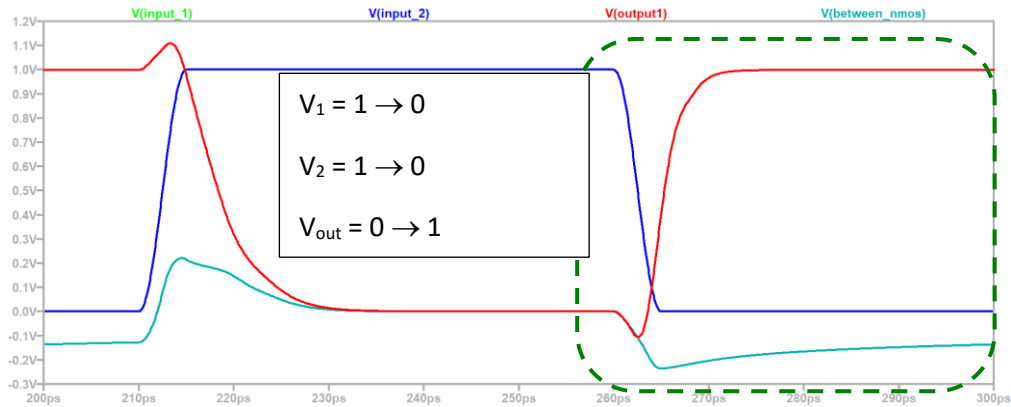


On the other hand, in the case $(1 \rightarrow 0, 1)$, the top NMOS transistor, and thus node X, are no longer connected to the output node. Hence, when the output voltage goes from 0 to 1, the only parasitic capacitance to be charged is the output capacitance, which requires less time than in the case $(1, 1 \rightarrow 0)$.



4. The value of t_{plh} in the case $(V_1 = 1 \rightarrow 0, V_2 = 1 \rightarrow 0)$ is significantly lower than in both $(V_1 = 1, V_2 = 1 \rightarrow 0)$ and $(V_1 = 1 \rightarrow 0, V_2 = 1)$ cases. We indeed have $t_{plh}(1 \rightarrow 0, 1 \rightarrow 0) \sim 2.8ps$ compared to $t_{plh}(1, 1 \rightarrow 0) \sim 5.7ps$ and $t_{plh}(1 \rightarrow 0, 1) \sim 4.4ps$.

The reason is that both PMOS transistors, that are placed in parallel, are now turned on, which results in the doubling of the current used for charging the parasitic capacitance present at the output node. Due to the doubling of the charging current, it takes much less time to charge this capacitance compared to cases where only one PMOS transistor is on.

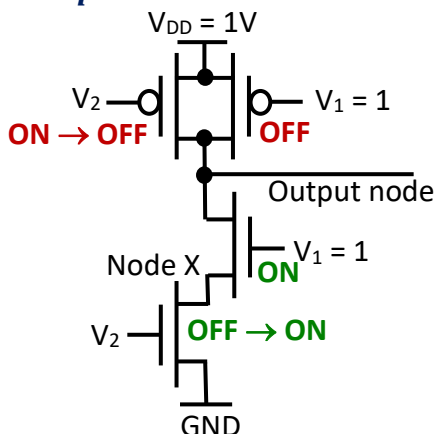


Finally, we create a simple table comparing the propagation times for a logic inverter and a two-input NAND gate. In all cases, all MOSFETs have $L = W = 45\text{nm}$ and the gate is loaded by a single inverter with the same size. In this case, the inverter and two-input NAND gate are said to have a unit fan-out.

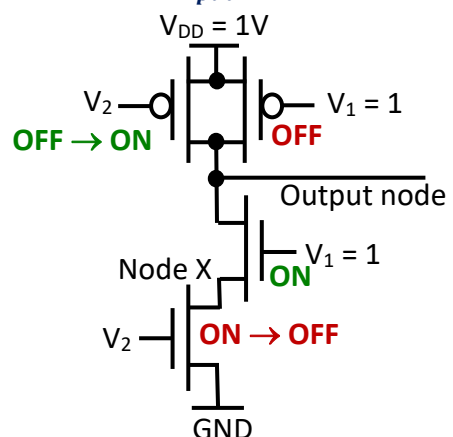
$L = W = 45\text{nm}$, fan-out = 1, input's rise and fall times = 5ps	Inverter	Two-input NAND gate	Worst-case input configuration*
t_{plh}	3.9ps	5.7ps	$V_1 = 1, V_2 = 1 \rightarrow 0$
t_{phl}	2.7ps	6.0ps	$V_1 = 1, V_2 = 0 \rightarrow 1$

* V_1 is the voltage applied on the gate of the NMOS closest to output node ("top NMOS"), whereas V_2 is the voltage applied on the gate of the NMOS closest to ground ("bottom NMOS").

Worst-case scenario for t_{phl} : Output node and node X must both be discharged $\rightarrow t_{phl} \sim 6.0\text{ps}$.



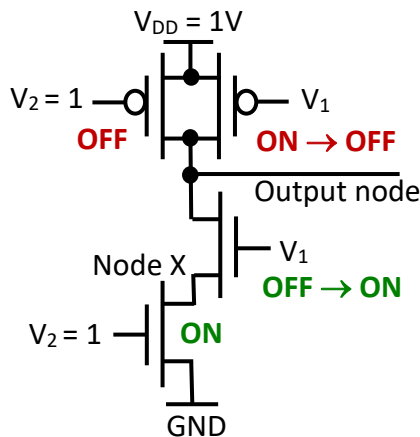
Worst-case scenario for t_{plh} : Output node and node X must both be charged through one PMOS only $\rightarrow t_{plh} \sim 5.7\text{ps}$.



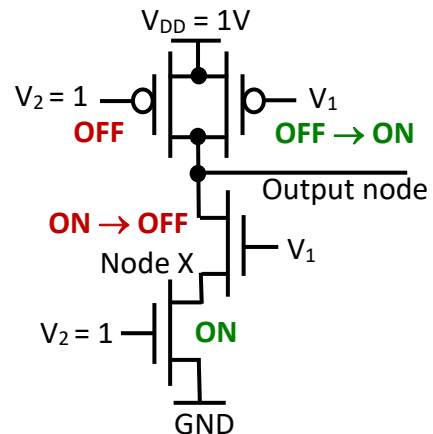
When designing a circuit using such two-input NAND gate, we have to assume the worst-case scenario for the propagation times to be on the safe side, i.e. use the numbers $t_{plh} \sim 5.7\text{ps}$ and $t_{phl} \sim 6.0\text{ps}$. However, if we know that one input signal will always be settled much before the other one, we can

connect this “early” input to the gate of the NMOS closest to ground, while connecting the “late” input to the gate of the NMOS closest to output node. In this case, we are allowed to use the numbers $t_{plh} \sim 4.4ps$ and $t_{phl} \sim 5.0ps$.

V_2 arrives before V_1 : Only output node must be discharged because node X has already been discharged before the switch in $V_1 \rightarrow t_{phl} \sim 5.0ps$.



V_2 arrives before V_1 : Only output node must be charged because node X is not connected to output node $\rightarrow t_{plh} \sim 4.4ps$.



Propagation times of NAND gates with more than two inputs

L = W = 45nm, fan-out = 1, input's rise and fall times = 5ps	Inverter	Two-input NAND gate	Three-input NAND gate	Four-input NAND gate	Five-input NAND gate	Input configuration
t_{plh}	3.9ps	5.7ps	7.2ps	8.4ps	9.4ps	1
t_{plh}		4.4ps	5.0ps	5.5ps	6.0ps	2
t_{plh}		2.8ps	2.6ps	2.4ps	2.2ps	3
t_{phl}	2.7ps	6.0ps	10.2ps	14.9ps	20.5ps	4
t_{phl}		5.0ps	7.5ps	9.5ps	12.7ps	5
t_{phl}		6.0ps	9.9ps	14.5ps	19.8ps	6

Configuration 1: All inputs = 1, except that of NMOS closest to ground which switches from 1 to 0.

Configuration 2: All inputs = 1, except that of NMOS closest to output which switches from 1 to 0.

Configuration 3: All inputs switch from 1 to 0 at the same time.

Configuration 4: All inputs = 1, except that of NMOS closest to ground which switches from 0 to 1.

Configuration 5: All inputs = 1, except that of NMOS closest to output which switches from 0 to 1.

Configuration 6: All inputs switch from 0 to 1 at the same time.

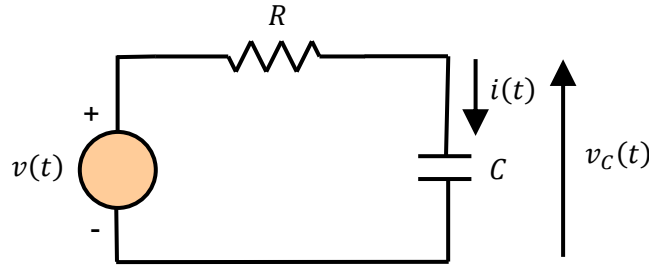
Conclusions: Configurations 1 and 4 are worst-case scenarios are thus to be considered by default. However, if we are certain that input signals settle at very different times, configurations 2 and 5 can be considered instead as they are more realistic and less penalising.

Appendix – Charge and discharge of a capacitance through a resistance

The charging process of a capacitance through a PMOS transistor is analogous to that occurring through a simple resistance. Also, the discharging process of a capacitance through an NMOS transistor is analogous to that occurring through a resistance.

Consider first the charge of a capacitance through a resistance in a classical circuit made up of a voltage source $v(t)$, a resistance R , and a capacitance C .

Before time $t = 0$, we assume that $v(t) = 0$. As a result, there is no current flowing through the circuit: $i(t) = 0$. The capacitance C is completely discharged, i.e. $q(t) = 0$, and we thus have $v_C(t) = \frac{q(t)}{C} = 0$, where $v_C(t)$ denotes the voltage across C .



Then, at time $t = 0$, the supply voltage is set to a positive value: $v(t) = V$. We know what happens then: the capacitance is progressively charged by a current $i(t)$ provided by the voltage source. At some stage, the capacitance C is fully charged and the current $i(t)$ goes back to zero. When this happens, the amount of charge stored in the capacitor is given by $Q = CV$.

In fact, before $t = 0$, the circuit was in a DC state in the sense that the current $i(t)$ and voltage $v(t)$ were both constant. Here, we had $i(t) = 0$ and $v(t) = 0$.

After $t = 0$, the circuit progressively goes to a new DC state, i.e. the current $i(t)$ and the voltage $v(t)$ ultimately take new DC values which are $i(t) = 0$ and $v_C(t) = V$.

However, the circuit does not go into this new DC state instantaneously at $t = 0$. Instead, there is a time window just after $t = 0$ during which the circuit is in a *transient state*. During that time, the values of both $i(t)$ and $v(t)$ keep on changing.

Using KVL, we can write that the following equation, valid at any time after $t = 0$:

$$Ri(t) + v_C(t) = V.$$

In addition, we know that the current $i(t)$ flowing through the capacitance and the voltage $v_C(t)$ across the capacitance are linked via the following expression:

$$i(t) = C \frac{dv_C(t)}{dt}.$$

By combining these two equations, we obtain a first-order differential equation:

$$RC \frac{dv_C(t)}{dt} + v_C(t) = V.$$

This differential equation has the following solution, valid for any $t \geq 0$:

$$v_C(t) = V + A \exp\left(-\frac{t}{RC}\right),$$

where A is a constant.

In theory, the constant A can take any value. However, we are here confronted with a practical problem rather than a theoretical one. So, we need to find the particular value of A that will make the previous expression compatible with reality.

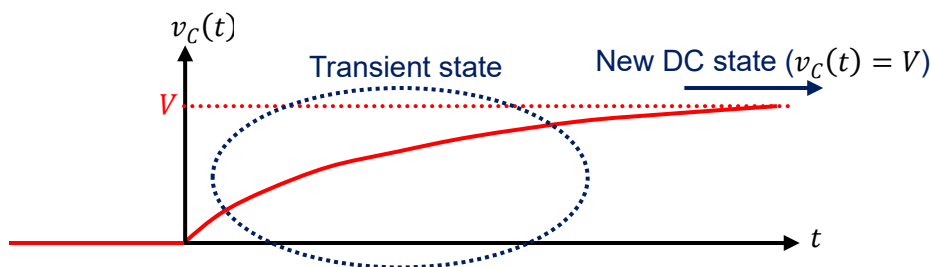
To do so, the general method consists of using the fact that, in the real (physical) word, there is always some particular quantity that remains constant at the (spatial and/or time) boundaries of the system.

Here, the boundary of our RC circuit is the time $t = 0$ because this is the time at which the value of the supply voltage changes. The quantity that must remain constant from $t = 0 - dt$ to $t = 0 + dt$, with $dt \rightarrow 0$, is the voltage $v_C(t)$ because the amount of charge $q(t) = Cv_C(t)$ stored in the capacitance cannot change instantaneously. In other words, we must ensure that $q(+dt) = q(-dt) = 0$, and thus $v_C(+dt) = v_C(-dt) = 0$.

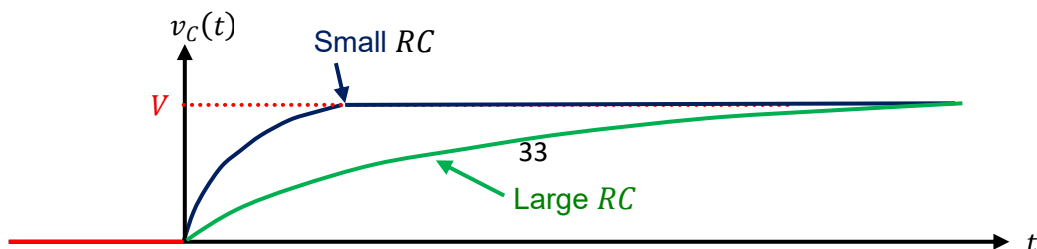
Therefore, the expression $v_C(t) = V + A \exp\left(-\frac{t}{RC}\right)$ only makes sense if it allows $v_C(0)$ to be equal to zero. It is easy to see that, according to this expression, $v_C(0) = V + A$. In order to have $v_C(0) = 0$, we must thus have $A = -V$.

We finally obtain the expression of $v_C(t)$, valid for any $t \geq 0$, i.e. for both the transient state and the new DC state:

$$v_C(t) = V \left(1 - \exp\left(-\frac{t}{RC}\right)\right).$$



It is worth mentioning that the amount of time needed to reach the new DC state after $t > 0$ depends on the value of the RC product, often referred to as the time constant. The smaller this time constant the shorter the duration of the transient phase.

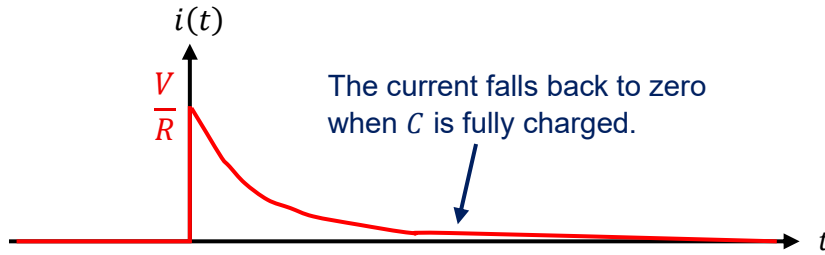


This can be understood by remembering that it takes more time to charge a larger capacitance (since more electrons have to be stored before full capacity is reached). At the same time, increasing the resistance value slows down the charging process, i.e. the resistance R acts as a “brake” in the charging process.

The RC circuit is a very important basic circuit in electricity as its analysis allows us to clearly understand why circuits that are very capacitive tend to respond to electric stimuli very slowly. This is why it is so crucial to minimise the parasitic capacitances in any electronic circuit that must operate at high frequencies, i.e. be able to quickly switch from one DC state to the next.

Finally, we can also find the expression of the current $i(t)$ flowing through the circuit as follows:

$$i(t) = C \frac{dv_C(t)}{dt} = C \frac{d}{dt} \left[V \left(1 - \exp \left(-\frac{t}{RC} \right) \right) \right] = -CV \frac{d}{dt} \left[\exp \left(-\frac{t}{RC} \right) \right] = \frac{V}{R} \exp \left(-\frac{t}{RC} \right).$$



Let us now consider the energy and power issues in this circuit.

(1) The total energy dissipated in the resistance during the charging process is given by

$$E_R = \int_0^T p_R(t) dt = \int_0^T i(t) v_R(t) dt = \int_0^T i(t) (V - v_C(t)) dt,$$

where

- T is an arbitrary time quantity chosen large enough so that the inequality $\exp \left(-\frac{T}{RC} \right) \ll 1$ is satisfied.
- $p_R(t)$ designates the instantaneous power dissipated by resistance R at time t .
- $v_R(t)$ denotes the voltage across R at time t .

We can thus write

$$E_R = V \int_0^T i(t) dt - \int_0^T i(t) v_C(t) dt = \frac{V^2}{R} \int_0^T \exp \left(-\frac{t}{RC} \right) dt - \frac{V^2}{R} \int_0^T \left(\exp \left(-\frac{t}{RC} \right) - \exp \left(-\frac{2t}{RC} \right) \right) dt,$$

which is equivalent to

$$E_R = \frac{V^2}{R} \int_0^T \exp \left(-\frac{2t}{RC} \right) dt = \frac{V^2}{R} \left[-\frac{RC}{2} \exp \left(-\frac{2t}{RC} \right) \right]_0^T = \frac{CV^2}{2} \left(\exp(0) - \exp \left(-\frac{2T}{RC} \right) \right).$$

As we have assumed that $\exp\left(-\frac{T}{RC}\right) \ll 1$, it is clear that we also have $\exp\left(-\frac{2T}{RC}\right) \ll 1$. So, finally, we obtain the expression of the total energy dissipated in the resistance during the whole charging process:

$$E_R = \frac{1}{2} CV^2.$$

(2) The total energy stored in the capacitance during the charging process is given by

$$E_C = \int_0^T p_C(t) dt = \int_0^T i(t) v_C(t) dt,$$

where $p_C(t)$ designates the instantaneous power stored in C at time t . We can thus write

$$E_C = \int_0^T i(t) v_C(t) dt = \frac{V^2}{R} \int_0^T \left(\exp\left(-\frac{t}{RC}\right) - \exp\left(-\frac{2t}{RC}\right) \right) dt,$$

which is equivalent to

$$E_C = \frac{V^2}{R} \left[-RC \exp\left(-\frac{t}{RC}\right) + \frac{RC}{2} \exp\left(-\frac{2t}{RC}\right) \right]_0^T.$$

This can be further developed as

$$E_C = \frac{V^2}{R} \left(-RC \exp\left(-\frac{T}{RC}\right) + \frac{RC}{2} \exp\left(-\frac{2T}{RC}\right) + RC \exp(0) - \frac{RC}{2} \exp(0) \right).$$

As we have assumed that $\exp\left(-\frac{T}{RC}\right) \ll 1$, it is clear that we also have $\exp\left(-\frac{2T}{RC}\right) \ll 1$. So, finally, we obtain the expression of the total energy stored in the capacitance during the whole charging process:

$$E_C = CV^2 \left(1 - \frac{1}{2} \right) = \frac{1}{2} CV^2.$$

(3) The total energy provided by the power supply to the RC circuit during the charging process is given by

$$E = \int_0^T p(t) dt = \int_0^T i(t) v(t) dt = V \int_0^T i(t) dt.$$

We can thus write

$$E = \frac{V^2}{R} \int_0^T \exp\left(-\frac{t}{RC}\right) dt = \frac{V^2}{R} \left[-RC \exp\left(-\frac{t}{RC}\right) \right]_0^T,$$

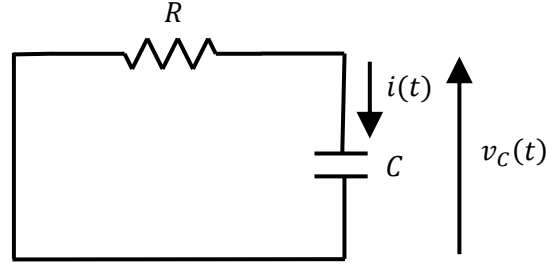
which is equivalent to

$$E = CV^2 \left(-\exp\left(-\frac{T}{RC}\right) + \exp(0) \right)$$

By using once more the inequality $\exp\left(-\frac{T}{RC}\right) \ll 1$, we obtain the expression of the total energy provided by the power supply to the RC circuit during the charging process:

$$E = CV^2.$$

Note that we have $E = E_C + E_R$, which is something that we expected. This simply means that half of the energy provided by the power supply, the latter being given by $E = CV^2$, is dissipated as heat by the resistance and the other half is stored in the capacitance.



We can also address the discharging process occurring if the voltage source $v(t)$ falls from V to 0 at a certain time $t = T$. In this case, we can write the following equation, valid at any time after $t = T$:

$$Ri(t) + v_C(t) = 0,$$

which leads to the following first-order differential equation:

$$RC \frac{dv_C(t)}{dt} + v_C(t) = 0.$$

This differential equation has the following solution, valid for any $t \geq T$:

$$v_C(t) = A \exp\left(-\frac{t}{RC}\right),$$

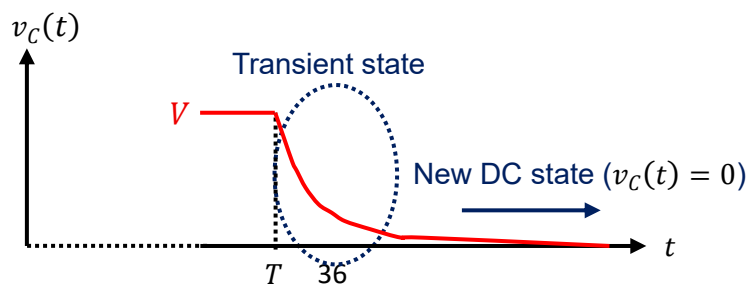
where A is a constant.

Here, the boundary of our RC circuit is the time $t = T$ because this is the time at which the value of the supply voltage changes. The quantity that must remain constant from $t = T - dt$ to $t = T + dt$, with $dt \rightarrow 0$, is the voltage $v_C(t)$ because the amount of charge $q(t) = Cv_C(t)$ stored in the capacitance cannot change instantaneously. In other words, from a physical viewpoint, we must have $q(T + dt) = q(T - dt) = CV$, and thus $v_C(T + dt) = v_C(T - dt) = \frac{CV}{C} = V$, with $dt \rightarrow 0$.

Therefore, the expression $v_C(t) = A \exp\left(-\frac{t}{RC}\right)$ only makes sense if it allows $v_C(T)$ to be equal to V . It is easy to see that, according to this expression, $v_C(T) = A \exp\left(-\frac{T}{RC}\right)$. In order to have $v_C(T) = V$, we must thus have $A = V \exp\left(\frac{T}{RC}\right)$.

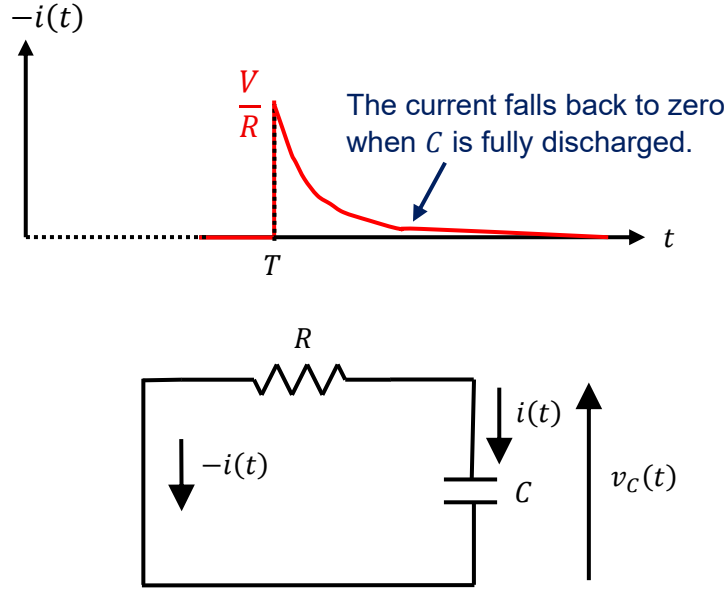
We finally obtain the expression of $v_C(t)$, valid for any $t \geq T$, i.e. for both the transient state and the new DC state:

$$v_C(t) = V \exp\left(\frac{T}{RC}\right) \exp\left(-\frac{t}{RC}\right) = V \exp\left(-\frac{t-T}{RC}\right).$$



Finally, we can also find the expression of the current $i(t)$ flowing through the circuit as follows:

$$i(t) = C \frac{dv_C(t)}{dt} = C \frac{d}{dt} \left[V \exp \left(-\frac{t-T}{RC} \right) \right] = -\frac{V}{R} \exp \left(-\frac{t-T}{RC} \right).$$



Let us now consider the energy and power issues in this circuit.

(1) The total energy dissipated in the resistance during the discharging process is given by

$$E_R = \int_T^{2T} p_R(t) dt = \int_T^{2T} i(t) v_R(t) dt = - \int_T^{2T} i(t) v_C(t) dt.$$

We can thus write

$$E_R = \frac{V^2}{R} \int_T^{2T} \exp \left(-2 \frac{t-T}{RC} \right) dt = \frac{V^2}{R} \left[-\frac{RC}{2} \exp \left(-\frac{2(t-T)}{RC} \right) \right]_T^{2T} = \frac{CV^2}{2} \left(\exp(0) - \exp \left(-\frac{2T}{RC} \right) \right),$$

which leads to

$$E_R = \frac{1}{2} CV^2.$$

(2) The total energy stored in the capacitance during the charging process is given by

$$E_C = \int_0^T p_C(t) dt = \int_0^T i(t) v_C(t) dt,$$

which leads to

$$E_C = -\frac{V^2}{R} \int_T^{2T} \exp \left(-2 \frac{t-T}{RC} \right) dt = -\frac{V^2}{R} \left[-\frac{RC}{2} \exp \left(-\frac{2(t-T)}{RC} \right) \right]_T^{2T} = \frac{CV^2}{2} \left(\exp \left(-\frac{2T}{RC} \right) - \exp(0) \right),$$

which yields

$$E_C = -\frac{1}{2} CV^2.$$

The fact that this energy is negative implies that charges leave the capacitance, which is to be expected as this is discharging process. In other words, these equations clearly indicate that all the energy previously stored in the capacitance is being dissipated as heat in the resistance during the discharging process.

As a conclusion, let us consider a full cycle of charge-discharge of a capacitance. By using the same notation as in the previous reasonings, one can state that the duration of the full charge-discharge process is equal to $2T$. This corresponds to a voltage source that varies in time from zero volt to a certain positive value V and back to zero volt. It is thus a square periodic signal with a frequency $f = \frac{1}{2T}$.

We have shown that, over a time period $2T$, the total energy provided by the voltage source to the circuit is given by $E = CV^2$, which thus corresponds to an average power $P = \frac{E}{2T} = fCV^2$. This is an important result as it links the power consumption of a circuit with its frequency of operation, output capacitance, which is in practice of a parasitic nature, and the maximum voltage value.

During the first half of the cycle, half of the energy is stored in the output capacitance, while the other half is dissipated in the resistance. During the second half of the cycle, the energy that was stored in the capacitance is released and dissipated in the resistance.

- END -